future internet

MDPI

Article

Lightweight Anomaly Detection in Digit Recognition Using Federated Learning

Anja Tanović and Ivan Mezei *

Faculty of Technical Sciences, University of Novi Sad, 21000 Novi Sad, Serbia; anja_tanovic@uns.ac.rs
* Correspondence: imezei@uns.ac.rs

Abstract

This study presents a lightweight autoencoder-based approach for anomaly detection in digit recognition using federated learning on resource-constrained embedded devices. We implement and evaluate compact autoencoder models on the ESP32-CAM microcontroller, enabling both training and inference directly on the device using 32-bit floating-point arithmetic. The system is trained on a reduced MNIST dataset (1000 resized samples) and evaluated using EMNIST and MNIST-C for anomaly detection. Seven fully connected autoencoder architectures are first evaluated on a PC to explore the impact of model size and batch size on training time and anomaly detection performance. Selected models are then re-implemented in the C programming language and deployed on a single ESP32 device, achieving training times as short as 12 min, inference latency as low as 9 ms, and F1 scores of up to 0.87. Autoencoders are further tested on ten devices in a real-world federated learning experiment using Wi-Fi. We explore non-IID and IID data distribution scenarios: (1) digit-specialized devices and (2) partitioned datasets with varying content and anomaly types. The results show that small unmodified autoencoder models can be effectively trained and evaluated directly on low-power hardware. The best models achieve F1 scores of up to 0.87 in the standard IID setting and 0.86 in the extreme non-IID setting. Despite some clients being trained on corrupted datasets, federated aggregation proves resilient, maintaining high overall performance. The resource analysis shows that more than half of the models and all the training-related allocations fit entirely in internal RAM. These findings confirm the feasibility of local float32 training and collaborative anomaly detection on low-cost hardware, supporting scalable and privacy-preserving edge intelligence.

Keywords: anomaly detection; TinyML; autoencoder; federated learning; TinyFL

Academic Editors: Suhardi Azliy Junoh and Jae-Young Pyun

Received: 2 July 2025
Revised: 26 July 2025
Accepted: 28 July 2025
Published: 30 July 2025

Citation: Tanović, A.; Mezei, I. Lightweight Anomaly Detection in Digit Recognition Using Federated Learning. *Future Internet* **2025**, *17*, 343. https://doi.org/10.3390/fi17080343

1. Introduction

The proliferation of machine learning (ML) on resource-constrained devices has gained significant momentum in recent years. This shift gave rise to a new paradigm known as TinyML [1], which focuses on deploying ML algorithms that are typically computationally intensive on tiny low-power devices such as microcontrollers (MCUs). The promise of enabling intelligent processing directly on edge devices has sparked considerable interest in the research community [2,3].

Recent advancements in edge computing have fueled the demand for intelligent autonomous systems that can operate in real time on resource-constrained hardware. In particular, applications such as surveillance, smart sensors, and industrial monitoring increasingly rely on on-device intelligence to reduce latency, preserve privacy, and minimize energy consumption. This subfield of edge computing has recently been referred to as

'extreme edge' [4], implying high resource constraints. A recently published article related to edge AI highlights the urgent need for power-efficient learning models that can be deployed on ultra-low-power hardware without sacrificing performance [5]. This is also important and aligns with the United Nations' 2030 Agenda for Sustainable Development and the demand for minimizing carbon emissions [6]. TinyML has its place in current wireless communication networks, but it will also have an important role in future networks [7].

In this context, anomaly detection (AD) plays a vital role in digit recognition and related classification tasks. Rather than requiring retraining to adapt to unseen or unusual data, anomaly detection systems can flag inputs that deviate from learned patterns—supporting fault detection, outlier filtering, or real-time quality control [8]. However, implementing robust anomaly detectors on embedded systems remains challenging due to the tight memory, compute, and power budgets of microcontrollers such as the ESP32-CAM.

To address these constraints, lightweight architectures such as simple autoencoders (AEs) have gained traction. These unsupervised models are well-suited for AD as they learn to reconstruct normal inputs and flag deviations by measuring reconstruction error. Prior work on TinyML and embedded AI emphasizes the need to minimize model size and maximize interpretability when designing such solutions for edge deployment [9,10].

At the same time, the growing concern for data privacy [11] has prompted the exploration of federated learning (FL) [12], a collaborative approach in which devices train models locally and share only model updates rather than raw data. This technique is especially attractive in scenarios involving sensitive data or distributed sensor networks [13]. Despite the growing interest in FL on tiny devices (TinyFL), the deployment of FL on highly constrained MCUs—particularly those with limited RAM and compute power, such as the ESP32—for on-device training remains underexplored [14].

Traditional TinyML applications focus primarily on deploying pretrained models for inference. However, emerging applications in FL require local training to preserve data privacy and reduce communication costs. In this study, we implement and evaluate a lightweight autoencoder directly on the ESP32-CAM microcontroller, performing both training and inference. By keeping all the data local to the device, we address privacy concerns that are central to distributed learning. Rather than optimizing memory through quantization or compression, we carefully characterize the available resources to determine feasible model sizes that can be trained and executed with full 32-bit floating-point precision. This retains model quality while enabling full on-device training and FL participation, demonstrating that collaborative learning is achievable even on resource-constrained MCUs.

We explore the intersection of anomaly detection, implementing lightweight models, and federated learning on embedded extreme edge devices. Our goal is to implement and evaluate an autoencoder-based AD system on an MCU, including FL scenarios with multiple real-world devices. Our main contributions are as follows:

- We design, implement, and evaluate several fully connected autoencoder models for digit-based anomaly detection, deployed fully on the ESP32-CAM microcontroller. Our approach enables on-device training and inference using standard 32-bit floating-point precision and incorporates an innovative dual-phase early stopping mechanism to optimize training efficiency. The best-performing models achieve an F1 score of up to 0.87 when optimized for MNIST and EMNIST anomaly detection, with on-device training times as short as 12 min and inference latency as low as 9 ms.
- We implement a real-world federated learning testbed across 10 ESP32-CAM devices and test various IID and non-IID data distributions, including corrupted datasets, to reflect realistic edge scenarios. Our extensive experimental results demonstrate

strong and consistent F1 scores, achieving up to 0.87 in the standard IID setting and 0.86 in the extreme non-IID setting.

- We analyze the robustness and resilience of FL in the presence of heterogeneous and corrupted data sources without relying on explicit malicious device detection. We evaluate scenarios where one to five out of the ten devices have corrupted datasets. The results show that federated aggregation effectively mitigates the impact of anomalous datasets, rendering their influence negligible for up to two corrupted devices and causing only minor performance degradation when more devices are affected.
- We characterize the ESP32-CAM's memory footprint by analyzing program size, dataset allocation, and memory consumption during training and inference to establish practical limits for embedded model deployment. The results demonstrate that more than half of the evaluated models fit entirely within the internal RAM, enabling faster training speeds while maintaining satisfactory performance.

The paper is organized as follows. Section 2 reviews the related work in the fields of anomaly detection, edge ML models, and federated learning. Section 3 presents the system architecture used throughout the experiments. Section 4 details the lightweight autoencoder deployment on the ESP32-CAM, including dataset preparation, model and training strategy, and embedded implementation. Section 5 introduces the federated training approach and evaluates performance under non-IID and IID data, including experiments on robustness to anomalous training data. Section 6 analyzes resource usage, focusing on memory footprint, dataset allocation, and training and model memory usage. Section 7 presents the experimental results on anomaly detection performance and on-device training feasibility, along with a comparison to the related work. Finally, Section 8 concludes the paper and outlines directions for future research.

2. Related Work

Anomaly detection has been extensively studied in recent years, with machine learning methods demonstrating strong performance across a range of domains. A comprehensive survey by De Albuquerque Filho et al. [15] highlights the success of neural network-based approaches, particularly autoencoders, which are the most frequently used for outlier detection. In a recent review, Ghamry et al. [16] provide a broad overview of anomaly detection techniques, covering both traditional ML and deep learning-based methods and outlining their suitability across various application domains. Various recent studies have further advanced the field by proposing novel AD techniques. Carannante et al. [17] propose a Bayesian deep learning framework that leverages model uncertainty to identify distributional shifts, achieving robust detection under noise, adversarial attacks, and label perturbations. Denouden et al. [18] enhance traditional autoencoder-based AD by incorporating Mahalanobis distance in the latent space, demonstrating improved out-of-distribution detection across varying bottleneck sizes. Meanwhile, Ruff et al. [19] explore anomaly detection as a classification problem, showing that, even with very few labeled anomalies, classifiers can effectively separate normal and anomalous data points. Complementing these works, Haider et al. [20] emphasize the growing relevance of edge computing in AD systems for future 6G-enabled IoT applications, where ultra-low latency and distributed intelligence will be essential.

As interest in deep learning grows, researchers are increasingly targeting embedded platforms and low-power hardware for running ML models directly on the edge. This shift has led to the rise of TinyML [1], which focuses on deploying ML models on constrained edge devices. A recent survey by Boroujeni et al. [21] emphasizes the multidisciplinary nature of TinyML, highlighting its critical role in modern industrial revolutions and its impact on applications such as smart cities and robotics. Similarly, Zeeshan et al. [22]

underline the growing need to move ML from the cloud to the edge, where IoT devices must process massive data streams with minimal latency and power usage.

To address the challenges of limited compute, memory, and energy resources on edge devices, various optimization techniques have been explored. For example, weight pruning and quantization methods have been proposed to reduce model size and energy usage during inference [23,24]. MCUNet [25] exemplifies this trend by combining neural architecture search (TinyNAS) with a memory-efficient runtime (TinyEngine), enabling ImageNet-scale inference on microcontrollers. Collectively, these efforts showcase the growing potential of deploying ML at the extreme edge using TinyML principles.

Autoencoders have become a widely adopted approach for anomaly detection on edge devices due to their unsupervised nature and low inference cost. Kim et al. [26] proposed a squeezed convolutional variational autoencoder, designed for real-time AD within Industrial IoT environments. Their approach prioritizes model compression and low-latency inference, making it suitable for edge deployment. Similarly, Givnan et al. [27] demonstrate autoencoder-based fault detection for rotary machines using real-time vibration data, enabling interpretable alerts for proactive maintenance. Bratu et al. [28] also explore this domain by deploying a compact AE directly on an ESP32 device, enabling continuous fully local monitoring of motor vibrations without requiring cloud connectivity. Moallemi et al. [29] deploy both fully connected and convolutional autoencoders on ultra-low-power STM32L4 MCUs for structural health monitoring, achieving efficient real-time inference at the edge while minimizing network load. Finally, Luo and Nagarajan [30] propose a hybrid approach in which AEs are used for local anomaly detection on wireless sensor nodes, while model updates are handled by the cloud at configurable intervals, reducing communication overhead in distributed IoT systems.

While the previously mentioned studies demonstrate the feasibility and efficiency of performing AD inference on edge devices, the majority of the existing work assumes that model training is performed offline, on more powerful hardware, before deployment. This approach stems from the limited computational, memory, and energy resources available on embedded platforms, which often make full training impractical. Nonetheless, recent studies have demonstrated that, with appropriate strategies, on-device training of deep neural networks is feasible even on highly resource-constrained MCUs. TinyTrain [31] introduces a task-adaptive sparse update mechanism that selectively fine-tunes layers or channels, significantly reducing memory and computation overhead while maintaining accuracy and fast training. Similarly, Deutel et al. [32] propose a fully quantized training framework with dynamic partial gradient updates, allowing efficient training on Cortex-M MCUs with substantial reductions in memory and latency. Focusing on unsupervised approaches, Ren et al. [33] present TinyOL, a system for online training of AEs directly on streaming data in IoT devices, supporting both supervised and unsupervised setups. Abbasi et al. [34] design a family of highly compact convolutional AEs called OutlierNets, with as few as 686 parameters, showing that accurate AD is possible with extremely lightweight models deployable on devices with limited RAM.

As edge intelligence advances, FL has become an increasingly popular strategy for enabling collaborative model training across distributed IoT devices without sharing raw data. This paradigm, often referred to as TinyFL when applied to MCUs and resource-constrained nodes, supports privacy preservation and decentralized learning. Qi et al. [35] provide a comprehensive survey of lightweight FL approaches, categorizing key techniques and identifying open challenges relevant to embedded deployments. Several recent works explore the feasibility of FL on edge platforms. For instance, Kopparapu et al. [36] present TinyFedTL, the first open-source implementation of FL on ultra-constrained IoT devices with <1 MB memory, showing that federated transfer learning is feasible and effective

even under severe memory and processing constraints. Ficco et al. [37] propose combining FL with transfer learning to enable on-board training for classification and regression tasks on IoT devices, evaluating performance against TensorFlow Lite-based solutions. Nikic et al. [38] demonstrate an end-to-end FL pipeline for digit recognition using a camera-equipped ESP32 MCU, highlighting that on-device training combined with FL can maintain high accuracy (97.01%) while keeping communication energy costs minimal. Ren et al. [39] further extend this direction with TinyReptile, a federated meta-learning approach for TinyML devices, enabling fast adaptation of neural networks on resource-constrained devices such as the Raspberry Pi 4 and Cortex-M4 MCU with only 256 KB of RAM.

Recent efforts have also explored combining FL with autoencoder-based AD on the edge. Novoa et al. [40] propose a deep autoencoder for FL, a privacy-preserving framework that reduces training time via non-iterative distributed training. Liu et al. [41] design a federated 5G-edge architecture for Industrial IoT, leveraging LSTM-based AEs for AD. Similarly, Reis et al. [42] introduce Edge-FLGuard, integrating AEs and LSTMs in a federated pipeline to enable low-latency on-device inference for cybersecurity in IoT networks. Olanrewaju et al. [43] evaluate both supervised and unsupervised DL models for intrusion detection, demonstrating that an FL-trained AE achieves the best performance across nine IoT devices using the N-BaIoT dataset. Ochiai et al. [44] present WAFL-Autoencoder, a fully distributed FL scheme based on device-to-device collaboration without centralized coordination.

3. System Architecture

The system consists of several ESP32-CAM edge devices and a central server hosted on a personal computer (PC). Each ESP32-CAM is equipped with a 240 MHz dual-core Xtensa processor, integrated Wi-Fi, and a built-in camera, making it suitable for low-power edge computing tasks such as local training and data collection [45]. The server is a standard desktop PC running Windows 10, equipped with an 11th Gen Intel Core i7-1165G7 CPU at 2.80 GHz and 12 GB of RAM, and it is responsible for global coordination of the federated learning process. Since the server does not perform intensive training but only coordination and model aggregation, it does not require a high-end GPU or multicore processing.

The architecture follows a star network topology where all ESP32-CAM devices communicate directly with the server over Wi-Fi. The server acts as the central node in the network, while ESP32 devices operate as clients. Figure 1 illustrates both communication phases: uplink (blue arrows), where ESP32-CAM devices transmit data to the server, and downlink (orange arrows), where the server sends data back to the devices.

The ESP32-CAM devices perform three key roles:

- Data collection: Capture sensor or image data.
- On-device training: Train lightweight ML models locally.
- Federated participation: Exchange model parameters with the server during the FL.

The server coordinates the system and manages communication by

- Training configuration: Sending configuration parameters to devices, such as the number of epochs, learning rate, and model structure.
- Model aggregation: Receiving local model updates from devices, aggregating them into a global model, and distributing the updated model.
- Model synchronization: Providing the latest global model to devices that rejoin the training process after disconnection or power loss.

Communication between ESP32-CAM devices and the server is handled over Wi-Fi using a lightweight HTTP protocol, with data formatted in JSON for simplicity and compatibility. A basic request–response synchronization is used, ensuring that devices wait

for updated configurations or model responses before proceeding to the next step. In case of temporary power loss or connectivity issues, devices can rejoin the network and request the current global model without disrupting or restarting the training process.

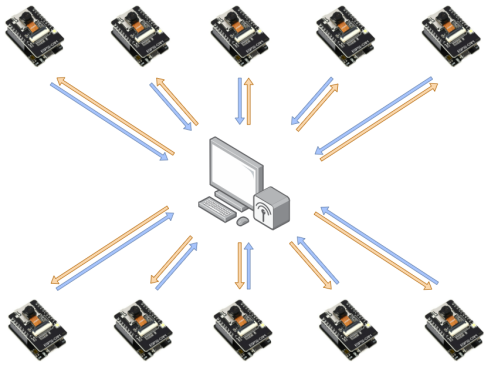


Figure 1. FL setup used in this study, illustrating communication between ESP32-CAM devices and a central server in a star topology over Wi-Fi. Each device performs local training and sends model updates to the server, which aggregates them and distributes the global model back to the devices.

4. Autoencoder Deployment on ESP32-CAM

This section presents the deployment of an autoencoder on the ESP32 MCU, covering dataset selection and modification, the model architecture and training strategy, and the embedded implementation, including on-device training. The following subsections provide a detailed description of each aspect.

4.1. Dataset Selection and Modification

To evaluate autoencoder-based anomaly detection on edge, we selected three datasets with varying visual complexity: MNIST [46], EMNIST [47], and MNIST-C [48], all accessed via the TensorFlow Datasets library [49]. The MNIST dataset contains grayscale images of handwritten digits (0–9) and serves as a standard benchmark for classification and reconstruction models. It was chosen for its simplicity, compact size, and widespread use in digit recognition tasks. EMNIST extends MNIST by including both uppercase and lowercase handwritten letters, introducing greater character diversity and task complexity. MNIST-C (Corrupted MNIST) enhances the original MNIST dataset by applying a range of algorithmic corruptions (e.g., noise, blur, translations, and contrast changes), simulating real-world visual distortions that may occur on embedded vision systems.

We used MNIST data for training and labeled it as normal, whereas EMNIST and MNIST-C were employed as anomalous datasets for testing. This setup allowed us to assess the autoencoder's effectiveness in detecting anomalies across all three datasets.

To accommodate the memory limitations of the ESP32-CAM, only 1000 grayscale images from the MNIST training set were used for training the autoencoder. For testing, 2000 images from the MNIST test set were employed to verify that the model correctly identifies normal MNIST digits and does not misclassify them as anomalies. Although the test set is unusually larger than the training set, this setup ensures a more reliable and robust evaluation of reconstruction performance. In real-world edge deployments, a separate test set is typically not available, and only locally collected training data exists. However, we include a dedicated test set here for controlled evaluation and analysis purposes.

To assess the model's ability to detect unfamiliar patterns, we evaluated it using 2000 images from the EMNIST dataset and four different variants from the MNIST-C dataset, each contributing an additional 2000 images. These datasets simulate out-of-distribution inputs that an edge device might encounter in real-world conditions.

To further reduce memory usage, all images were resized to 14 × 14 pixels and normalized to the [0,1] range. Additionally, we did not utilize a separate validation set. Instead, the MNIST training set was used for both training and validation-related purposes. This choice reflects the constraints of real-world embedded systems, where memory resources are limited, and using the full dataset for training can improve model performance.

While MNIST is a publicly available dataset, we use it to simulate the type of digit data an edge device would collect locally from a user. In real-world federated settings, each device gathers and trains on its own private data, which remains local. Although actual data collected by edge devices may differ, MNIST provides a representative baseline for digit recognition tasks. Building on this, the AE architecture was selected through preliminary experiments on the MNIST dataset, demonstrating the feasibility of training a lightweight AE on the ESP32-CAM and preparing it for realistic FL scenarios.

4.2. Model Architecture and Training Strategy

We implemented and evaluated seven small-scale fully connected AE architectures suitable for embedded deployment. Each model consists of an encoder and decoder surrounding one or more hidden layers that encode a latent representation of the input. The input consists of 14 × 14 grayscale images flattened to 196 features. The encoder compresses this input to a lower-dimensional latent space, and the decoder reconstructs it back to the original input size. All models use ReLU as the activation function, selected for its simplicity and efficiency, and the Adam optimizer with a learning rate of 0.001, which is commonly used for autoencoders due to its fast convergence. We evaluated multiple models with varying numbers of hidden layers and neurons to determine the best balance between model complexity, reconstruction performance, and resource constraints. The model architectures are as follows:

- Model 1: Single hidden layer with 16 neurons—architecture: `[196, 16, 196]`.
- Model 2: Single hidden layer with 32 neurons—architecture: `[196, 32, 196]`.
- Model 3: Single hidden layer with 64 neurons—architecture: `[196, 64, 196]`.
- Model 4: Two hidden layers with 32 neurons each—architecture: `[196, 32, 32, 196]`.
- Model 5: Three hidden layers with a narrow bottleneck of 8 neurons—architecture: `[196, 32, 8, 32, 196]`.
- Model 6: Three hidden layers with a narrow bottleneck of 16 neurons—architecture: `[196, 64, 16, 64, 196]`.
- Model 7: Five hidden layers with 32-neuron bottlenecks and wider 64-neuron outer layers—architecture: `[196, 64, 32, 32, 64, 196]`.

In on-device training scenarios, where data is collected locally and its characteristics are unknown beforehand, it is challenging to predefine the optimal number of training epochs. Combined with the strict power and time constraints of edge devices, this requires mechanisms to avoid unnecessary computation. To address this, we implemented a dual-phase early stopping strategy, inspired by TensorFlow's EarlyStopping mechanism [50], which monitors training or validation loss and stops training when no significant improvement (defined by *min delta*) is observed over a specified number of epochs (*patience*). Our approach extends this concept by using two different *patience* values for two training phases. Initially, in the first phase, the model is allowed to train with a higher patience value (*patience phase1*) until a predefined *acceptable loss* threshold is reached. After this point, a lower patience value (*patience phase2*) is applied, allowing the training to terminate quickly when further improvements are minimal (smaller than *min delta*). This mechanism is particularly suitable for on-device training scenarios, where the optimal number of epochs cannot be known in advance and both training time and energy consumption must be minimized.

Unlike typical early stopping strategies that save and restore the best model after training—often requiring the storage for multiple checkpoints—our approach is optimized for memory-constrained embedded devices. Since saving earlier model states is impractical on such platforms, the second phase uses a very low patience value (typically below 5) combined with an appropriate *min delta* threshold. This setup ensures that, once the loss reaches an acceptable level in the first phase and stops improving significantly in the second phase, training is halted immediately. This prevents potential degradation that could occur if training continued through a plateau or local minimum. As a result, the final model is effectively "frozen" at a satisfactory point, without risking performance loss or requiring rollback, making the process suitable for training on embedded devices.

AD was performed by measuring the reconstruction error using mean squared error (MSE) between the input and the reconstructed output image as the MSE provides a standard and effective metric for quantifying reconstruction discrepancies in image data. Samples with a reconstruction error that exceeded a defined threshold were classified as anomalies. This anomaly threshold was set using the 95th percentile of reconstruction errors computed on the training data, effectively treating 95% of the training samples as normal and allowing up to 5% to be considered potential outliers.

As previously mentioned, no separate validation set was used during training. Instead, the training set was used for tasks typically handled by a validation set, such as loss evaluation, anomaly threshold calculation, and early stopping check. This choice was driven by the strict memory constraints of edge devices, where maximizing the size of the training set can improve generalization. While using a dedicated validation set is certainly possible and may be preferable in less constrained environments, reusing the training data allows all available memory to be allocated to the training set or the model, avoiding the overhead of reserving memory for validation.

The models were initially implemented and trained on a PC using Python 3.10.0 and the TensorFlow 2.17.0 library [51]. This allowed us to experiment with different configurations and evaluate model behavior under various conditions. Each architecture was trained using multiple batch sizes (1, 10, and 50) to analyze their impact on performance, particularly in preparation for deployment on memory-constrained edge devices. Model evaluation was conducted using several performance metrics, including final training loss (measured by MSE), training and inference time, AD rate (defined as the percentage of test samples flagged as anomalous), and the F1 score based on AD results. After evaluation, a suitable training configuration was selected for each model based on batch size and performance trade-offs.

4.3. Embedded Deployment and Training

To bring the AE model to a memory-constrained embedded platform, we re-implemented it in the C programming language and deployed it on the ESP32-CAM. The model used 32-bit floating-point values (float32) for all model parameters, including weights, biases, and neuron activations. This mirrors the TensorFlow implementation on PC, which also defaults to float32 for the above parameters, although it may internally use higher precision (e.g., float64) for certain operations. As a result, minor differences in output between the PC and embedded versions are expected due to precision handling. Aside from the model, other parts of the embedded system, such as loop counters, control flags, and class labels, used the smallest possible integer types to minimize memory consumption.

To support a wide range of training scenarios and deployments, we implemented a reconfigurable model architecture within a single firmware. Instead of hard-coding a specific AE design, the ESP32-CAM establishes a Wi-Fi connection to a central server, which provides a complete training configuration, including the model architecture, early stopping

parameters, activation functions, optimizer type, and other training-related parameters. Upon receiving this configuration, the device dynamically allocates memory and constructs the model. This approach allows the same firmware to adapt to various experimental and operational needs without requiring firmware reflashing.

As mentioned in the previous subsection, all models were designed to fit within the available memory of the ESP32-CAM, prioritizing internal RAM for speed. However, when RAM alone is insufficient, parts of the model and training buffers are dynamically allocated in external pseudostatic RAM (PSRAM). This hybrid allocation ensures that larger models can still be executed without failure while retaining performance benefits where possible. In low-power scenarios such as deep-sleep modes, the model can optionally be saved to Flash and reloaded when needed.

For evaluation purposes, test datasets were stored on the SD card. However, in real-world inference deployments, the SD card is not required. During actual usage, input data typically arrives in real time from on-board sensors (e.g., camera or environmental sensors). Thus, there is no need to preload a static test set from external storage. Only RAM and PSRAM are necessary for live inference and on-device learning.

We compared the training loss and AD performance of models implemented on the ESP32-CAM with their TensorFlow counterparts on a PC. By initializing the models with the same random weight values, we ensured a consistent starting point for comparison. Although the ESP32-CAM showed slightly lower accuracy for larger models due to the previously mentioned precision, performance trends remained consistent, validating the robustness of the embedded implementation. A full performance comparison—such as training time, inference time, and AD quality—is presented in the chapter dedicated to experimental results and analysis.

5. Federated Training of Edge Autoencoders

To evaluate FL on constrained embedded platforms, we implemented a real-world experimental testbed involving ten ESP32-CAM devices and a central server connected via Wi-Fi, as described in the System Architecture section. Each device was equipped with an SD card to store its local training and test sets and run the same model architecture and identical hyperparameters. However, the training data was partitioned differently across devices to reflect data diversity and imbalance often encountered in real-world deployments. This setup enabled us to evaluate FL performance directly on actual hardware, with a focus on convergence behavior under non-IID (non-identically independently distributed) conditions and the system's robustness to corrupted or anomalous data in IID (identically independently distributed) conditions.

To facilitate communication and model aggregation, a Wi-Fi server acts as the central node in the federated system. Training is organized in alternating rounds of local training and federated aggregation. After each training round, devices send their updated models to the server, which aggregates them using federated averaging (FedAvg) [52] and redistributes the resulting global model to all participants for the next round.

We adopted the anomaly threshold computation method proposed in [44], which enables devices to collaboratively determine a global anomaly threshold via device-to-device communication. Specifically, we used the formula (5) from [44], where each device computes a global threshold α^n by aggregating the local thresholds β^n, and received thresholds α^k from neighbors:

$$\alpha^n \leftarrow \frac{\sum_{k \in \mathrm{nbr}(n)} \alpha^k + \alpha^n + \gamma \beta^n}{|\mathrm{nbr}(n)| + 1 + \gamma}. \quad (1)$$

Here, α^n represents the global anomaly threshold for device n, and β^n is its locally calculated threshold based on training data (e.g., using 95th percentile metric). In our setup, we do not perform true peer-to-peer communication; instead, devices send their values to a central server, which mimics neighbor aggregation. Therefore, the number of neighbors is fixed to $|\text{nbr}(n)| = 10$ for all devices, and we use a weighting coefficient of $\gamma = 0.1$.

Unlike the model, which is synchronized across devices, the anomaly threshold remains device-specific. This aggregation method allows each device to retain a threshold that reflects its local data distribution while incorporating information from the broader network. As a result, thresholds remain slightly biased toward local conditions, which improves their suitability for detecting anomalies in each device's context. This balance is especially useful after additional rounds of local fine-tuning, where a personalized threshold performs better than a purely averaged one.

Additionally, to ensure efficiency and avoid unnecessary computation, we employ a previously described dual-phase early stopping mechanism, adapted for FL. While early stopping is still applied at the local level during each device's training phase (as in the standalone setup), we additionally apply early stopping at the global level for federated training. After each round, when devices receive the updated global model and complete the following local training phase, they evaluate their performance and vote on whether to terminate the training. A device votes to stop if the model's performance has not improved for a predefined number of rounds. In our implementation, we adopt the strictest configuration with *patience phase2* set to 0, meaning that even a single round without improvement triggers the voting. If the majority of devices vote to stop, the FL process is terminated, conserving both communication and energy resources once convergence has been effectively reached.

We explored two federated learning scenarios:

- **Scenario 1 (Non-IID—Single Digit per Device):** Each device was assigned images of a single digit from MNIST—e.g., device 0 trained on digit '0', device 1 on digit '1', and so on—covering all ten digit classes. This setup represents an extreme case of non-IID data distribution.
- **Scenario 2 (IID—Partitioned Dataset):** Each device received a distinct subset of the MNIST dataset containing all ten digit classes but with no overlap between subsets. This scenario investigates two aspects:
  - Training on Distinct MNIST Subsets: We evaluated how well the global model learns when devices are trained on disjoint partitions of the dataset.
  - Robustness to Anomalous Training Data: Selected devices were provided with corrupted training samples instead of regular MNIST, allowing us to assess the global model's resilience in the presence of local data contamination.

The following subsections provide detailed descriptions of these federated training scenarios.

5.1. Non-IID—Single Digit per Device

This experimental scenario is designed to evaluate the limits of federated learning under an extreme non-IID data distribution. Each of the ten ESP32-CAM devices is assigned training data corresponding to only a single digit class from the MNIST dataset; for instance, device 0 trains solely on images of digit '0', device 1 on digit '1', and so on. To create these device-specific datasets, we iterated through the MNIST training set of 10,000 samples and extracted only the entries matching the target label for each device. For example, to prepare data for the device assigned digit '0', only samples labeled as '0' were selected and stored in a separate file. This process is summarized in Algorithm 1. Due to the roughly uniform class distribution, each device received approximately 1000 training samples, stored in

PSRAM, which aligns with the sample size used in our prior experiments. As a result, no device saw a complete view of the dataset during local training.

Algorithm 1 Non-IID Partitioning: Single Digit per Device

Require: MNIST dataset (X, Y) with 10,000 samples
for each digit label $d = 0$ to 9 **do**
  Initialize empty list `selected_samples`
  for each sample (x_i, y_i) in (X, Y) **do**
    if $y_i = d$ **then**
      Append (x_i, y_i) to `selected_samples`
    end if
  end for
  Save `selected_samples` as a new dataset for the device assigned digit d
end for

While this setup is not typical in real-world applications, it represents a critical worst-case scenario that tests the collaborative power of federated learning. It investigates whether multiple small isolated learners, each exposed to only a narrow slice of the input space, can collectively produce a global model that generalizes across all classes.

This non-IID scenario has been explored in previous works [18,44] using larger more complex AEs. Differently, our approach focuses on edge deployment by implementing smaller resource-efficient models and leveraging a dual-phase early stopping strategy to minimize training epochs and communication rounds. This makes our method well-suited for low-power devices like the ESP32 while preserving effective performance.

5.2. IID—Partitioned Dataset

In this scenario, the MNIST dataset was evenly divided among devices such that each device received a distinct subset containing all digit classes in roughly equal proportions. This represents an IID setting where the local datasets closely resemble the overall distribution. Within this setup, we investigate two aspects: the effect of training on distinct subsets and the model's robustness when some devices are trained on corrupted versions of their data.

5.2.1. Training on Distinct MNIST Subsets

In this experimental setup, we investigate a more typical IID scenario where each ESP32-CAM device is assigned a distinct subset of the MNIST dataset. The data is partitioned such that each device receives 1000 samples containing all digit classes (0–9) in approximately equal proportions. For example, device 0 receives the first 1000 images, device 1 the next 1000, and so on. This ensures that, although the specific samples differ, the class distribution remains balanced across devices.

Unlike the extreme non-IID case described earlier, this setup reflects a more realistic federated learning environment in which devices independently collect diverse but representative data. This experiment allows us to evaluate how well a lightweight AE model can learn meaningful global features when trained on individually balanced but disjoint local datasets. Since the data distributions are representative and overlapping, we expect faster convergence and improved generalization compared to non-IID scenarios.

5.2.2. Robustness to Anomalous Training Data

To assess the robustness of FL in the presence of anomalous data, we extend the previous IID scenario by introducing corrupted training samples to a subset of devices. Specifically, while most ESP32-CAM devices train on standard MNIST subsets, one or more selected devices are assigned fully corrupted datasets from MNIST-C, which con-

tains anomalous versions of digits with distortions such as blur, noise, fog, and other environmental effects.

This setup simulates realistic deployment conditions, where certain edge devices may collect low-quality or distorted data due to environmental factors. For example, a device operating outdoors might gather anomalous data during sudden weather changes like fog or rain. If such a device trains solely on these corrupted data, its local model may lead to poor global performance.

In contrast to approaches that attempt to detect or exclude such anomalous clients, our study evaluates how much natural resilience standard FL provides without relying on additional defense mechanisms. Most existing FL robustness research focuses on identifying and mitigating the influence of malicious clients [53], often through techniques such as gradient clipping, client selection, or outlier detection, which aim to reduce their impact on the global model or exclude them from the FL process (e.g., [54–56]). But there is less emphasis on understanding how much anomaly exposure a federated model can naturally tolerate without intervention. Our work fills this gap by empirically evaluating how standard FL setups handle scenarios where certain clients are trained entirely on anomalous data. This allows us to assess whether clean models from unaffected devices can outweigh the influence of corrupted ones during aggregation, and whether corrupted clients can improve through federated aggregation.

6. Resource Utilization Analysis

The ESP32-CAM was selected as the target hardware platform due to its integrated Wi-Fi, dual-core processor, embedded camera, and compatibility with lightweight ML frameworks. Deploying both training and inference of an AE model, along with Wi-Fi-based communication for FL, required careful resource management within the device's constraints.

The ESP32-CAM features the Xtensa dual-core 32-bit LX6 processor running at up to 240 MHz. It includes 520 KB of internal SRAM, divided into 200 KB of IRAM (Instruction RAM) for storing executable code and 320 KB of DRAM (Data RAM) used for dynamic data. The module also includes external PSRAM (4 MB in this implementation) and 4 MB of Flash memory. These resource constraints significantly influence how models are stored, how training is executed, and how data is managed.

Training the AE directly on the ESP32-CAM was enabled by designing compact network architectures with reduced depth and neuron count. Integer arithmetic was employed wherever possible to reduce memory consumption and increase execution speed, while model weights were kept in floating-point format to maintain accuracy comparable to standard TensorFlow implementations.

6.1. Program and Memory Footprint

The firmware occupies approximately 933 KB of Flash memory, which includes compiled program code and constant data. Static RAM usage includes about 20 KB of DRAM for global variables and roughly 114 KB of IRAM for high-speed executable code. These values confirm that the firmware, including support for training, inference, communication, and configuration, fits within the device's memory limits.

The implementation includes dynamic runtime features, such as configurable model selection, optimizer choice, and AD thresholds. While the base AE architecture is lightweight, these added functionalities contribute to the program's overall memory usage. This additional complexity supports a flexible and reusable system that can handle a range of real-world scenarios rather than being a simple or limited demonstration.

6.2. Dataset Allocation in PSRAM

The training dataset is stored in external PSRAM due to its larger capacity and faster access compared to internal Flash or SD cards, as demonstrated in our previous work [57]. Because of limited memory, we used a reduced MNIST dataset containing 1000 images, each resized to 196 pixels. Stored as normalized 32-bit floats, the image data occupies approximately 784 KB (1000 × 196 × 4 bytes). Dataset labels, used here solely for experimental evaluation, add about 1 KB (1000 × 1 byte). In practical deployment, these would not be stored as data is collected in an unsupervised manner.

6.3. Model and Training Memory Usage

As discussed earlier, models are designed to run within the ESP32-CAM's available memory, with a preference for internal RAM to maximize speed. When RAM is insufficient, model parameters and training buffers are automatically allocated in external 4 MB PSRAM. Memory usage increases with model complexity as training requires additional space for activations, gradients, temporary buffers, and optimizer states.

Let the autoencoder have k layers, indexed from 0 to $k-1$, with layer sizes $L = [n_0, n_1, \ldots, n_{k-1}]$, where n_i is the number of neurons in layer i. The number of parameters P, including weights and biases, is calculated by summing over all pairs of adjacent layers. Therefore, the total number of parameters P is

$$P = \sum_{i=0}^{k-2} \left(\underbrace{n_i \times n_{i+1}}_{\text{weights}} + \underbrace{n_{i+1}}_{\text{biases}} \right) = \sum_{i=0}^{k-2} n_{i+1}(n_i + 1). \tag{2}$$

Each parameter is stored as a 32-bit float (4 bytes), so the memory required to store the model is

$$\text{Inference Memory (bytes)} = 4 \times P = 4 \times \sum_{i=0}^{k-2} n_{i+1}(n_i + 1). \tag{3}$$

This memory footprint is sufficient for inference-only applications, where no training is performed on the device. However, during training, additional memory allocations are required, including

- **Activations (*A*)**: Neuron values from the forward pass must be stored for backpropagation:
$$A = \sum_{i=0}^{k-1} n_i. \tag{4}$$
- **Gradients:** Gradients of the loss are accumulated during backpropagation. This requires the same number of float values as the model parameters: P floats.
- **Neuron activation function derivative buffers:** During backpropagation, these buffers temporarily store the derivatives of the activation functions for each neuron. Their size matches the number of neurons in each hidden and output layer:
$$D = \sum_{i=1}^{k-1} n_i. \tag{5}$$
- **Optimizer state (Adam):** Two additional floats per parameter (first and second moment estimates), totaling $2P$ floats.

  The total estimated memory for training is therefore

$$\text{Training Memory (bytes)} = 4 \times (P + A + P + D + 2P) = 4 \times (4P + A + D). \tag{6}$$

These equations allow the approximation of RAM consumption for each architecture during training. Table 1 summarizes the memory usage for both inference and training across all tested models. For inference, the smallest model (Model 1) required only 25 KB, while the largest (Model 7) reached nearly 120 KB. Training memory usage varied more significantly due to additional buffers and optimizer overhead, where the smallest model required 104 KB and the largest 482 KB.

Table 1. Summary of model size and total training memory usage on the ESP32 platform.

Model	Architecture	Inference Memory (KB)	Training Memory (KB)
Model 1	[196, 16, 196]	25.33	103.73
Model 2	[196, 32, 196]	49.89	202.11
Model 3	[196, 64, 196]	99.02	398.86
Model 4	[196, 32, 32, 196]	54.02	218.86
Model 5	[196, 32, 8, 32, 196]	52.05	211.05
Model 6	[196, 64, 16, 64, 196]	107.33	432.73
Model 7	[196, 64, 32, 32, 64, 196]	119.52	481.86

Careful memory tracking was essential to ensure that on-device training could operate within the available DRAM. Out of the 320 KB DRAM, about 20 KB is reserved for static allocations, leaving roughly 300 KB available for dynamic allocation of model parameters and training buffers. According to Table 1, all models fit within DRAM when used for inference only. However, only Models 1, 2, 4, and 5 fit entirely in DRAM during on-device training. Larger models (3, 6, and 7) exceed this capacity and rely on external PSRAM. When PSRAM is used, the system dynamically prioritizes DRAM for smaller allocations while transparently placing larger memory blocks, such as model weights or training buffers, in PSRAM when needed. This flexible allocation strategy enables training and inference of larger models without manual memory management. PSRAM was also used to store the dataset, freeing more DRAM for training operations. Despite resource constraints, the ESP32-CAM successfully deployed this model for on-device training and inference, proving its capability as a reliable and low-power platform for local training and edge-based FL.

7. Experimental Results and Analysis

To evaluate the effectiveness of lightweight AE models for AD in digit recognition, we conducted a series of experiments on both PC and ESP32-CAM platforms. The models were first tested on a desktop environment to assess the training dynamics, AD accuracy, and the impact of batch size and model complexity. Based on these findings, the selected models were ported to the ESP32 and evaluated in terms of runtime performance and AD quality.

Beyond a standalone evaluation, we implemented a real-world FL testbed using multiple ESP32-CAM devices and a central server communicating over Wi-Fi. We explored two data distribution scenarios (IID and non-IID) to simulate common deployment conditions. The results are presented in the following subsections.

Table 2 summarizes the key parameters and configurations that remain consistent throughout all the experiments. These include the learning settings, early stopping configurations, and AD threshold strategy, with some specific to FL and others for local training.

7.1. Evaluation of AutoencoderPerformance on PC

Before deploying autoencoder models to embedded devices, we evaluated seven architectures on a PC to understand how model complexity and batch size influence both training efficiency and AD performance. All the models were trained on the MNIST

using TensorFlow, with three different batch sizes: 1, 10, and 50. The goal was to explore whether larger models offer significantly better anomaly detection quality and whether such improvements justify the associated resource overhead. The primary performance metric was the F1 score, while training loss and time were tracked as secondary indicators.

Table 2. Configuration parameters used throughout all test scenarios.

Parameter	Value	Usage
Activation function	ReLU	Defines neuron output behavior
Optimizer	Adam	Adaptive weight updates
Learning rate	0.001	Step size for updates
Loss function	MSE	Measures prediction error
Patience phase 1	20	Dual-phase early stopping
Patience phase 2	3	Dual-phase early stopping
Acceptable loss	0.02	Dual-phase early stopping
Min delta	0.0005	Dual-phase early stopping
AD threshold function	95th percentile	AD thresholding
gamma	0.1	AD thresholding (FL)
Patience phase 1 (FL)	20	Dual-phase early stopping (FL)
Patience phase 2 (FL)	0	Dual-phase early stopping (FL)
Acceptable loss (FL)	0.02	Dual-phase early stopping (FL)
Min delta (FL)	0.0005	Dual-phase early stopping (FL)

As shown in Table 3, we report the number of epochs to convergence (based on a dual-phase early stopping criterion), final training loss, total training time (in seconds), inference time (in milliseconds), AD rates for all the evaluation sets, and F1 scores, which were used to select the best-performing batch size for each model.

We observed that some larger models, particularly when trained with higher batch sizes, failed to converge, triggering early stopping due to a lack of improvement in training loss. Models with a final training loss above the defined acceptable loss threshold of 0.02 are considered not converged and can be identified in the results table by their loss values. This indicates that greater model complexity or larger batch sizes do not necessarily lead to better performance and may instead inhibit effective learning. On the other hand, smaller batch sizes (especially batch size 1) led to longer training times due to reduced computational efficiency, while larger batches benefited from TensorFlow's internal optimizations.

To assess each model's effectiveness in AD, we evaluated their performance using the MNIST test set (in-distribution), four MNIST-C variants (fog, motion blur, dotted line, and spatter), and the EMNIST dataset (out-of-distribution). In our setup, images from the MNIST dataset were treated as normal data, while samples from the EMNIST and MNIST-C datasets represented anomalies. The goal was to optimize the model to minimize both types of misclassification: false positives (incorrectly flagging MNIST digits as anomalies) and false negatives (failing to detect EMNIST and MNIST-C as anomalies). Ideally, the model should identify anything that is not a digit as anomalous, making the F1 score a suitable metric. Since distinguishing digits (MNIST) from letters (EMNIST) is the most relevant and challenging task (due to the structural differences between letters and digits), we focused on optimizing the F1 score specifically for EMNIST. Table 3 highlights the best F1 score for each model across all the batch sizes. These configurations were selected for deployment and testing on the embedded platform, as described in the next subsection.

Figure 2 further illustrates AE behavior, showing the distribution of AD rates across datasets for the first model trained with various batch sizes. An ideal behavior is characterized by low detection rates for MNIST (test set) and high for EMNIST and MNIST-C. As the figure shows, increasing the batch size leads to higher rates across all the datasets, indicating a shift toward higher recall at the cost of precision. This trade-off reinforces our use of the F1 score to select the most balanced configuration.

Table 3. Autoencoder training performance and anomaly detection results on MNIST, MNIST-C, and EMNIST (on PC).

Model Number	Training Configuration			Training Results			Anomaly Detection Rate (%)						F1 Score
	Autoencoder Architecture	Batch Size	Epochs	Final Training Loss	Training Time (s)	Inference Time (ms)	MNIST	MNIST-C Fog	MNIST-C Motion Blur	MNIST-C Dotted Line	MNIST-C Spatter	EMNIST	
1	[196, 16, 196]	1	11	0.0159	13.67	1.06	9.55	90.70	6.85	33.40	14.55	75.70	0.8173
		10	26	0.0122	4.96		10.40	93.30	9.05	48.00	18.75	83.45	0.8610
		50	60	0.0124	4.50		11.60	93.65	10.60	55.05	21.20	85.90	**0.8699**
2	[196, 32, 196]	1	13	0.0109	16.30	1.13	8.50	91.90	6.35	25.55	12.05	69.25	0.7792
		10	29	0.0065	5.10		13.90	98.15	21.30	81.50	33.05	85.90	0.8599
		50	61	0.0074	4.57		17.35	97.95	23.90	87.90	41.30	88.75	**0.8612**
3	[196, 64, 196]	1	16	0.0087	20.07	1.24	7.50	92.15	6.80	24.20	10.60	66.45	0.7640
		10	30	0.0040	5.37		11.40	99.05	26.80	90.10	37.10	85.15	**0.8664**
		50	53	0.0057	4.15		14.40	98.15	21.80	89.25	36.15	84.95	0.8523
4	[196, 32, 32, 196]	1	32	0.0122	68.04	1.31	9.60	92.90	7.65	36.10	15.20	75.80	0.8177
		10	58	0.0119	18.01		14.10	95.00	14.70	63.20	26.00	86.35	0.8616
		50	124	0.0146	12.49		12.55	91.70	12.60	51.30	21.25	86.80	**0.8708**
5	[196, 32, 8, 32, 196]	1	48	0.0188	134.42	1.33	11.85	88.40	9.95	38.80	18.90	86.95	0.8747
		10	94	0.0191	30.97		12.05	89.20	10.30	41.55	17.80	89.80	**0.8898**
		50	83	0.0437	6.36		6.35	63.20	0.80	15.55	4.75	51.30	0.6508
6	[196, 64, 16, 64, 196]	1	37	0.0139	105.63	1.38	12.55	90.75	12.85	42.40	21.55	87.25	0.8734
		10	75	0.0145	24.79		15.70	93.80	20.00	56.55	28.65	90.25	0.8764
		50	164	0.0191	15.16		11.30	88.70	11.40	37.60	16.25	90.25	**0.8956**
7	[196, 64, 32, 32, 64, 196]	1	47	0.0169	141.89	1.34	17.05	91.35	19.15	48.40	26.15	93.20	0.8866
		10	107	0.0183	27.75		12.50	89.30	12.50	42.15	19.05	90.30	**0.8905**
		50	115	0.0424	9.18		7.20	67.80	1.30	18.85	6.20	57.70	0.6998

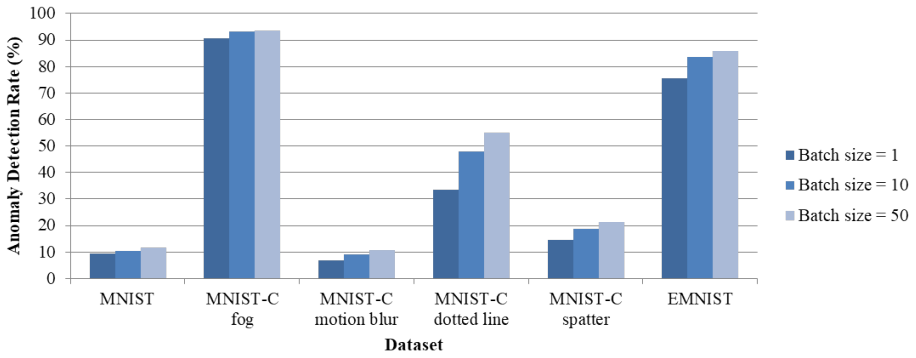


Figure 2. Anomaly detection rate distribution for different datasets using Model 1 `[196, 16, 196]` across batch sizes 1, 10, and 50.

To better understand how the AE distinguishes between normal and anomalous inputs, we visualized input and output image pairs for different datasets. Figure 3 shows examples from the MNIST, EMNIST, and MNIST-C test sets. For normal digit inputs, the AE successfully reconstructs the images with high fidelity, indicating that it has learned the underlying structure of the digits. In contrast, inputs from EMNIST or MNIST-C produce degraded reconstructions, resulting in higher reconstruction errors, which are interpreted as AD rates.

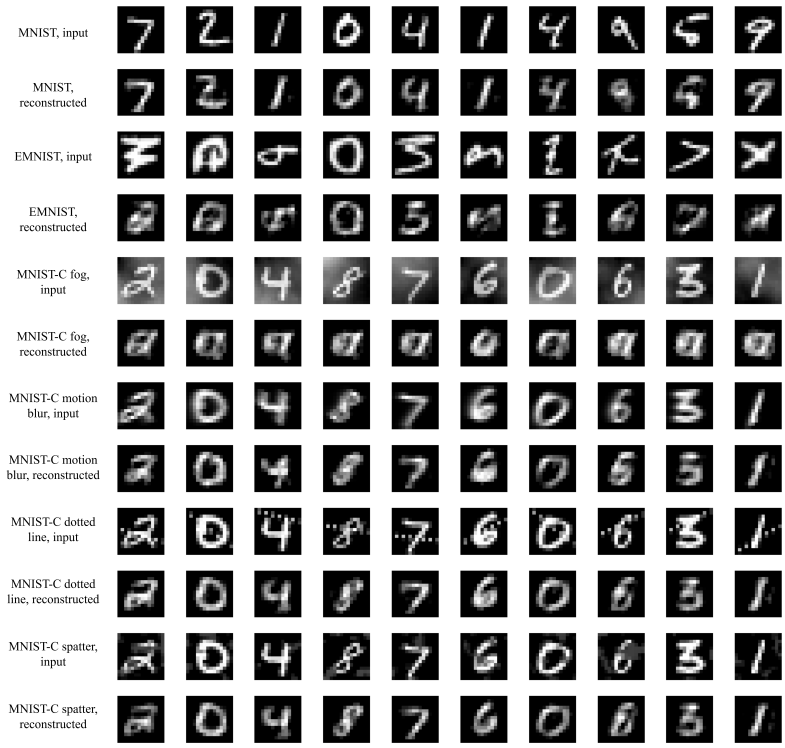


Figure 3. Input and reconstruction examples on autoencoder Model 1 `[196, 16, 196]`.

As shown in Figure 3, the MNIST digits are reconstructed almost perfectly since the model has been trained specifically on this distribution. The reconstructed shapes closely match the inputs, with only slight differences in clarity, resulting in low reconstruction errors that are not flagged as anomalies. In contrast, EMNIST samples, which were not seen during training, are usually unsuccessfully reconstructed. These significant differences lead to high reconstruction errors and, consequently, high AD rates. However, AD on EMNIST is not perfect due to certain letters resembling digits in structure, such as “O”, “I”, “L”, and “C”. As a result, some EMNIST characters are partially reconstructed and not

flagged as anomalies. This phenomenon limits the achievable F1 score on EMNIST, even for well-trained models.

This behavior can be explained by the nature of the learned latent space. Even though the AEs are trained only on digits, they often begin to capture more general visual features—such as strokes, loops, and corners—that are shared across both digits and some alphabetic characters. Over time, the model may learn to abstractly represent these components, allowing it to partially reconstruct letter-like inputs as well. Figure 4 illustrates the results from testing the first AE model, showing AD rates across EMNIST letter classes. Letters that are structurally similar to digits are detected less frequently as anomalies, making them harder to detect reliably.

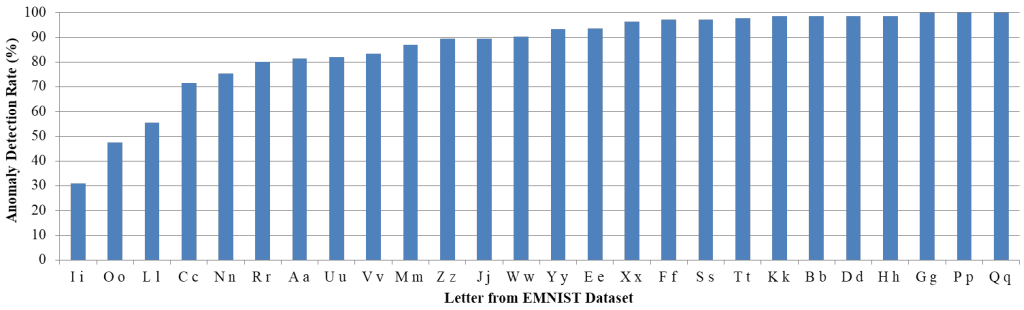


Figure 4. Sorted anomaly detection rate for each letter in EMNIST dataset for autoencoder Model 1 `[196, 16, 196]`.

Interestingly, when evaluating the MNIST-C dataset, we observe different behaviors. As illustrated in Figure 3, in the fog variant, global distortion leads to severe reconstruction failure, causing the AE to misinterpret the input entirely and correctly assign it a high AD rate. On the other hand, localized corruptions such as dotted line, spatter, or motion blur preserve most of the original digit shape. The model reconstructs the underlying digit relatively well while removing or ignoring the added noise during reconstruction. This demonstrates the AE's ability to perform implicit denoising, which is a useful byproduct of its training objective. However, the AD rates in such cases remain elevated since the model compares the corrupted input with its clean output and detects the difference. This highlights the sensitivity of reconstruction error: even if the digit is preserved, the presence of artifacts increases the AD rate.

These observations align with the results in Table 3, where EMNIST and MNIST-C fog consistently yield high F1 scores, confirming that the AE effectively flags them as anomalous. Other corrupted variants, being structurally closer to clean MNIST, result in more variable detection outcomes.

7.2. Evaluating the Dual-Phase Early Stopping Mechanism

To validate the advantages of the proposed dual-phase early stopping strategy, we conducted an additional experiment on PC that compared our strategy with the standard early stopping method using fixed patience values. This supplementary analysis demonstrates how the dual-phase approach effectively balances training efficiency and model performance.

For this analysis, we selected Model 4 with a batch size of 50 as a representative example. The standard early stopping was implemented using TensorFlow's callback, with a consistent *min delta* of 0.0005 and varying patience values (3, 10, 15, and 20). In contrast, our dual-phase strategy combines two patience levels: 3 for the initial phase and 20 once an acceptable loss threshold is reached. Figure 5 illustrates the training loss curves for both standard early stopping and the proposed dual-phase approach.

Figure 5a–d illustrate the training loss behavior using the standard strategy with increasing patience. As expected, higher patience values (e.g., 15 and 20) allow longer training and convergence, resulting in a stable loss plateau. Lower patience values (3 and 10) often terminate training prematurely, before the model escapes a local minimum, resulting in suboptimal final loss values.

While higher patience enables better convergence, it also prolongs training with diminishing returns in terms of loss improvement. On the other hand, smaller patience values reduce training time but risk early stopping before meaningful convergence. The dual-phase approach offers a compromise between these extremes. As shown in Figure 5e, it allows sufficient exploration early in training, followed by faster termination once the model reaches an acceptable performance threshold.

Table 4 summarizes the number of epochs and corresponding F1 scores across all the settings. The results indicate that extended training does not significantly improve F1 score and may even slightly degrade performance. In edge scenarios with limited power and time, the dual-phase strategy provides an efficient and effective trade-off between training cost and model quality.

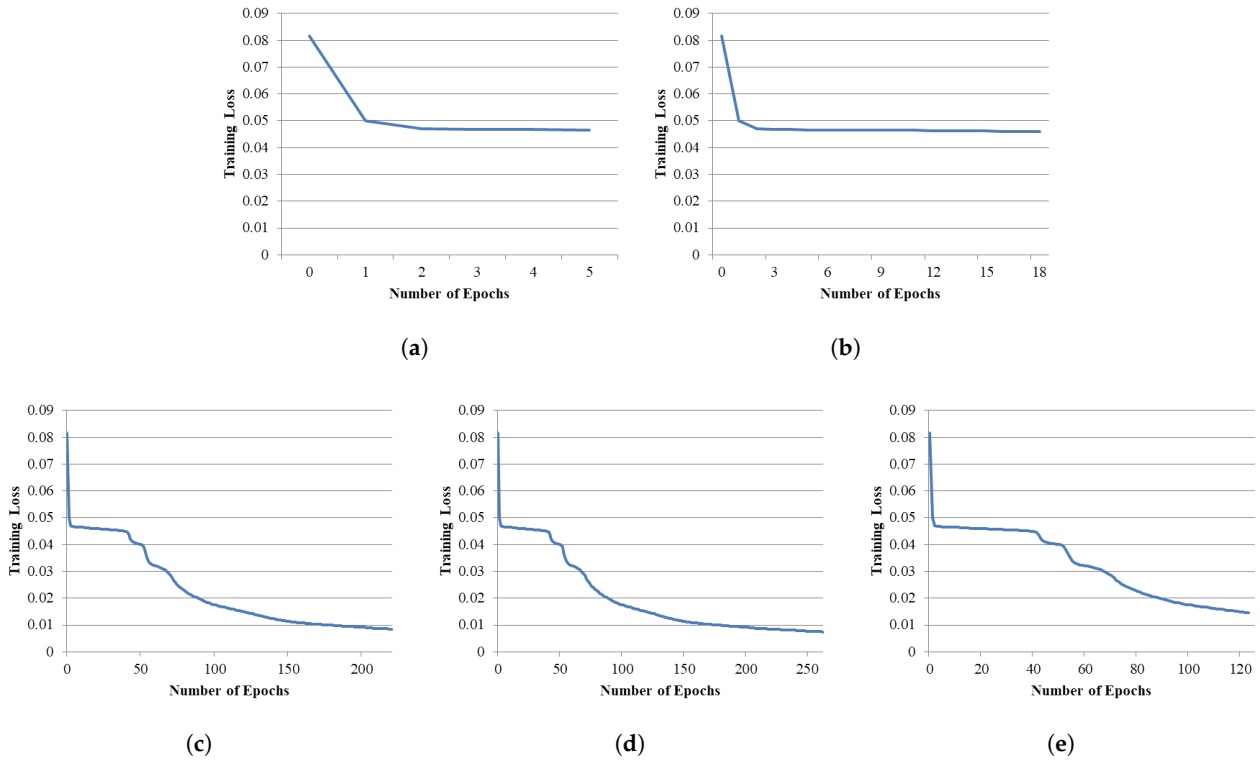


Figure 5. Training loss curves for Model 4 using different early stopping strategies: (**a**) standard early stopping with patience 3, (**b**) patience 10, (**c**) patience 15, (**d**) patience 20, and (**e**) proposed dual-phase early stopping with patience 3 for first phase and 20 for second phase.

Table 4. Comparison of early stopping strategies on anomaly detection performance.

Strategy	Epochs	Anomaly Detection Rate (%)		F1 Score
		MNIST	EMNIST	
Standard (patience = 3)	6	6.25	47.00	0.6134
Standard (patience = 10)	19	6.45	48.05	0.6220
Standard (patience = 15)	221	19.25	91.30	0.8673
Standard (patience = 20)	263	21.45	91.75	0.8607
Dual-phase (3/20)	124	12.55	86.80	0.8708

7.3. Evaluation of Autoencoder Performance on ESP32-CAM: Embedded Deployment

After evaluating all the AE models on a PC using TensorFlow, the next step was to deploy and test them on the ESP32-CAM to assess their behavior in a constrained embedded environment. Each of the seven selected autoencoder architectures was implemented in the C programming language and run with the previously chosen batch size.

To ensure consistency, we reused the same configurations from the PC evaluation on the ESP32, including the model architectures, datasets, dual-phase early stopping parameters, and F1 score calculation methodology. Furthermore, to ensure a proper comparison between the PC and embedded results, each ESP32-CAM training run was initialized with the same weights and biases as on the PC, ensuring the models followed an identical optimization path.

Table 5 summarizes the results for all seven models, including the model architecture, batch size, number of epochs until convergence (based on a dual-phase early stopping criterion), final training loss, total training time, inference time, AD rates, and F1 score. These results are then compared against their PC-trained counterparts.

The overall performance of the deployed models on ESP32 closely mirrors that of the PC. The only notable exception is the seventh model (the largest), which fails to converge due to exceeding the acceptable training loss threshold (0.02) during the first phase of the dual-phase early stopping mechanism. This is likely due to the increased number of computations required by deeper architecture, which leads to greater error accumulation in backpropagation under limited floating-point precision.

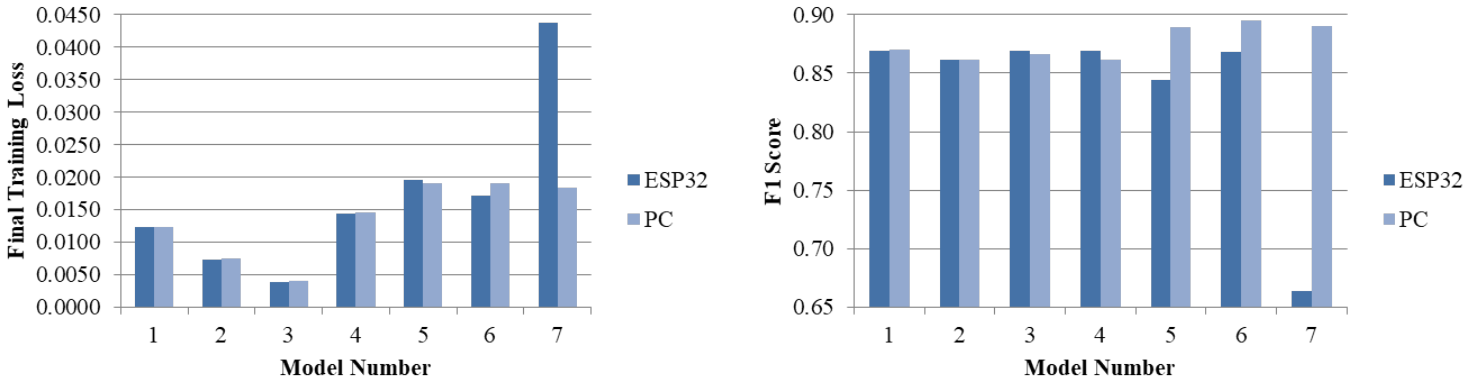
For all the other models, the training loss values and convergence trends remain consistent across the platforms. Figure 6 shows the final training loss values and F1 scores for each model on PC and ESP32-CAM. Figure 6a illustrates that the loss values overlap significantly (except for Model 7), indicating minimal deviation during training. Figure 6b compares the F1 scores and confirms that the embedded models achieve nearly the same anomaly detection accuracy as their PC-trained counterparts. The differences in performance slightly increase with deeper architectures, which again can be attributed to the accumulation of computational errors in backpropagation.

Although the training time on the ESP32-CAM is significantly longer (e.g., over 40 min for the third model compared to a few seconds on the PC), the learning dynamics remain consistent. Inference performance follows a similar trend: the third model requires approximately 32 ms per inference on the ESP32-CAM compared to 1.24 ms on a PC. While this difference is not negligible, it remains acceptable for applications that do not require high-throughput inference.

Despite the ESP32-CAM's limited processing power and 32-bit floating-point precision compared to higher-precision PC computations, the tested AE models achieved F1 scores up to 0.87, closely matching the PC performance. Among the seven models, all except the seventh showed similar AD accuracy, although training time increased significantly with model size. As Table 5 highlights, Model 4 attained the highest F1 score, but its advantage over Models 1, 3, and 6 is minimal. Given that shallower models train faster and require less memory, smaller architectures provide the most practical balance between performance and training feasibility on constrained embedded devices.

Table 5. Autoencoder training performance and anomaly detection results on MNIST, MNIST-C, and EMNIST (on ESP32-CAM).

Model Number	Training Configuration			Training Results			Anomaly Detection Rate (%)						F1 Score
	Autoencoder Architecture	Batch Size	Epochs	Final Training Loss	Training Time (s)	Inference Time (ms)	MNIST	MNIST-C Fog	MNIST-C Motion Blur	MNIST-C Dotted Line	MNIST-C Spatter	EMNIST	
1	[196, 16, 196]	50	60	0.0124	727.18	9.34	11.50	93.65	10.30	54.30	20.95	85.65	0.8689
2	[196, 32, 196]	50	61	0.0074	1720.17	16.41	17.10	97.95	23.55	87.60	40.90	88.55	0.8612
3	[196, 64, 196]	10	30	0.0038	2547.34	32.21	11.95	99.10	27.25	91.20	37.90	86.00	0.8689
4	[196, 32, 32, 196]	50	115	0.0144	3558.55	21.49	11.35	91.65	11.25	46.45	18.75	85.65	**0.8695**
5	[196, 32, 8, 32, 196]	10	111	0.0196	3577.33	16.47	12.30	88.20	7.85	33.35	15.15	81.95	0.8438
6	[196, 64, 16, 64, 196]	50	158	0.0172	12618.32	35.48	12.30	88.80	11.20	42.35	18.10	86.10	0.8679
7	[196, 64, 32, 32, 64, 196]	10	42	0.0437	4281.15	42.07	6.90	62.60	0.85	14.55	5.05	53.10	0.6638



(**a**) (**b**)

Figure 6. Performance comparison between PC and ESP32-CAM across all autoencoder models: (**a**) final training loss; (**b**) F1 score in anomaly detection.

7.4. Federated Learning Scenario 1: Non-IID—Single Digit per Device

To evaluate FL with the implemented AEs, it is important to select representative models that balance AD quality, training and inference speed, and resource usage. Since all seven candidate models exhibited comparable performance in AD, several criteria could be used to guide model selection. For instance, one might prioritize

- Training time—to reduce energy consumption and improve convergence speed during FL rounds.
- Inference time—if real-time detection is critical.
- Memory footprint—for devices with severe RAM/Flash limitations.

In our experiments, we opted to prioritize training efficiency as FL involves multiple rounds of local training on constrained hardware. Since all the models demonstrated comparable AD performance, choosing the fastest models in terms of training time allows us to minimize energy consumption and reduce the duration of each FL round. Thus, we selected the three models with the shortest training time during ESP32-based evaluation from the previous subsection. The selected models are as follows:

- Model 1: Single hidden layer with 16 neurons—architecture: `[196, 16, 196]`.
- Model 2: Single hidden layer with 32 neurons—architecture: `[196, 32, 196]`.
- Model 3: Single hidden layer with 64 neurons—architecture: `[196, 64, 196]`.

Each model undergoes multiple FL rounds, interleaved with local training phases. The training progression follows this sequence: *Training 0* $\rightarrow$ *Federated 0* $\rightarrow$ *Training 1* $\rightarrow$ *Federated 1* $\rightarrow \ldots \rightarrow$ *Federated n* $\rightarrow$ *Training n*$+1$, where n is determined by the dual-phase early stopping mechanism adapted for FL, as previously described. All early stopping parameters used at the global level were kept consistent with the local early stopping configuration, except for the second-phase patience, which was set to a stricter value of zero. This means that even a single round without improvement was sufficient to trigger the voting for stopping.

For all three selected models, convergence was consistently reached after two federated rounds (*Federated 0 and Federated 1*). This means that, after receiving the global model for the second time and performing additional local training, the majority of devices voted to stop training. Following that, each device sent its local model and stop decision back to the server. Although the stopping condition was met, we performed one final aggregation round for evaluation purposes, bringing the total to three federated rounds (*Federated 3*).

To maintain consistency, all the training configurations were kept the same as in the previous experiment. The only difference was that the number of training epochs was limited for each model during every local training phase. Specifically, Model 1 and Model 2 were each trained for 20 epochs, while Model 3 was trained for 10 epochs. These values were selected based on the results from the previous experiment, where the full duration of training was evaluated for each model. We used approximately one-third of the total number of epochs required to reach convergence in standalone training on the ESP32. This strategy allows each model to gain meaningful weight updates while still keeping training incomplete for federated learning.

The F1 scores, measured after each phase, are presented in Figure 7, illustrating how AD performance evolves per digit across training and federated rounds. This scenario represents a highly non-IID setting, where each ESP32-CAM device is trained exclusively on a single digit class from the dataset. After the initial local training round (*Training 0*), the F1 scores varied significantly across devices depending on the digit each device was trained on. However, after the first federated aggregation (*Federated 0*) was applied, the scores became more aligned across the devices, with only slight differences caused by the locally determined anomaly thresholds.

Following the second local training phase (*Training 1*), performance generally improves across all digits, especially with Model 3. The second federated round (*Federated 1*) results in the highest overall F1 scores for all three models, indicating that this is the optimal aggregation point. After receiving the second global model, devices proceed with another local training phase (*Training 2*) and then evaluate performance. During this phase, several devices observe a decline in performance, which activates the global early stopping mechanism. As the majority votes to terminate training, this signals that additional federated updates are unlikely to provide further benefit. A final federated round (*Federated 2*) was conducted to confirm this behavior, and the results showed a clear drop in performance, validating that *Federated 1* indeed represents the peak in collaborative learning.

Among all the tested architectures, Model 3 outperforms the others, achieving an average F1 score of approximately 0.86 after the *Federated 1* round. This result is very close to the 0.87 F1 score obtained in the centralized setting using the same model (Table 5). This gap is negligible considering the extreme data skew and distributed nature of training, highlighting the robustness of federated learning even in highly non-IID conditions.

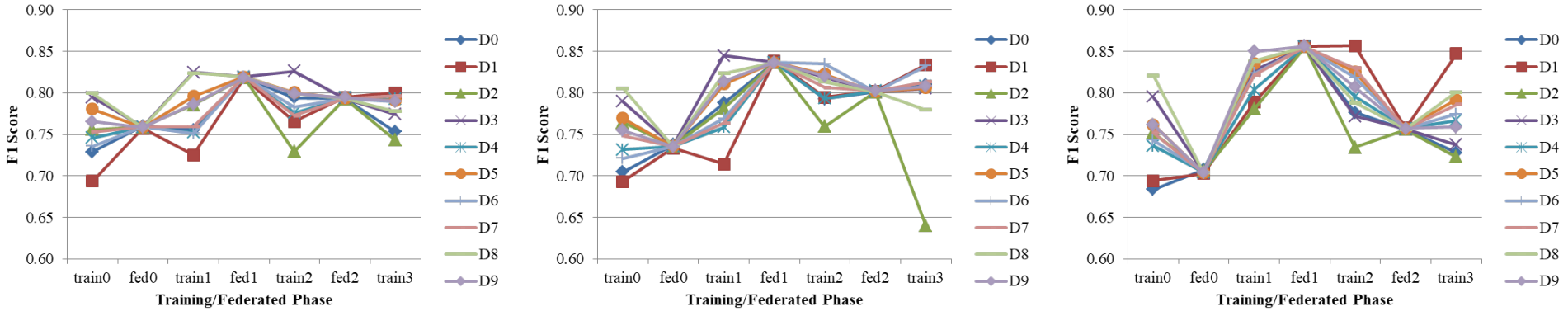
To further illustrate the AD capability, Table 6 presents the AD rates for each digit on both the MNIST and EMNIST datasets using Model 3 after FL. As shown in the table, the false positive rate for MNIST is approximately 9%, while the true negative rate on EMNIST is about 81.5% across all devices. These results are reasonable considering the simplicity of the model and the similarity between MNIST and EMNIST. The table also includes F1 scores along with their standard deviations across the devices, showing that the variation is larger after local training rounds compared to federated rounds, which aligns with expectations.

While not directly comparable, our experimental setup shares similarities with the study by Ochiai et al. [44], where each device is also trained on a single digit class from the MNIST dataset. Their model achieves a nearly perfect false positive rate of 1% compared to our 9% result. However, their approach differs in several important ways. They use a more complex AE architecture, perform model aggregation after every training epoch, and train on the full MNIST dataset of 60,000 samples (split by class). In contrast, we employ a significantly smaller autoencoder, use only 10,000 MNIST samples (also class-separated), and aggregate less frequently, after every 10 to 20 epochs, depending on the model, which results in a total of approximately 40 to 80 training epochs. Our design offers considerable advantages in terms of resource efficiency: reduced training time, significantly lower memory requirements, and reduced communication overhead by avoiding per-epoch aggregation. In constrained embedded environments, such a trade-off between performance and efficiency can be both practical and justified.

7.5. Federated Learning Scenario 2: IID—Partitioned Dataset

For the IID experimental scenario, the MNIST dataset was partitioned such that each ESP32 device received a subset containing samples from all the digit classes in approximately equal proportions. Under this configuration, we evaluated how well the models perform when trained on balanced but non-overlapping subsets, as well as the system's resilience to data-quality degradation by introducing artificially corrupted data on a subset of devices.

In the following experiments, the training procedure follows the same federated learning protocol as before: each device trains locally for a set number of epochs, after which the models are averaged on a central server. This process repeats until convergence, using the same early stopping configuration as in the previous scenario.



(**a**) (**b**) (**c**)

Figure 7. F1 score trends during federated training measured on devices trained with a single digit for (**a**) Model 1 `[196, 16, 196]`, (**b**) Model 2 `[196, 32, 196]`, and (**c**) Model 3 `[196, 64, 196]`.

Table 6. Anomaly detection performance per digit for Model 3 `[196, 64, 196]`.

Training/ Federated Phase	Anomaly Detection Rate (%)																				F1 Score $\pm\sigma$
	Digit 0		Digit 1		Digit 2		Digit 3		Digit 4		Digit 5		Digit 6		Digit 7		Digit 8		Digit 9		
	MNIST	EMNIST	MNIST	EMNIST	MNIST	EMNIST	MNIST	EMNIST	MNIST	EMNIST	MNIST	EMNIST	MNIST	EMNIST	MNIST	EMNIST	MNIST	EMNIST	MNIST	EMNIST	
train0	84.90	96.05	88.20	100.00	59.60	96.15	48.85	98.20	68.05	97.85	59.05	97.80	65.00	97.50	64.25	99.05	35.50	94.35	61.65	99.45	0.75 ± 0.0410
fed0	82.40	99.80	84.35	99.85	83.20	99.85	83.70	99.85	83.70	99.85	83.85	99.85	83.30	99.85	83.85	99.85	83.75	99.85	83.90	99.85	0.70 ± 0.0013
train1	15.45	81.30	50.25	97.80	7.15	68.60	7.35	75.60	28.40	86.30	23.80	88.75	24.10	86.85	25.45	87.45	12.90	81.50	18.20	87.35	0.82 ± 0.0220
fed1	**8.95**	**81.45**	**9.30**	**81.80**	**8.95**	**81.45**	**9.05**	**81.50**	**9.10**	**81.60**	**9.10**	**81.60**	**9.05**	**81.50**	**9.05**	**81.50**	**9.05**	**81.50**	**9.10**	**81.70**	**0.86 ± 0.0003**
train2	4.05	66.05	17.45	88.00	3.15	59.85	3.00	64.70	7.40	70.95	9.15	76.65	4.45	72.35	9.35	77.55	5.75	68.80	5.60	71.45	0.80 ± 0.0349
fed2	1.80	61.85	1.90	62.15	1.80	61.85	1.80	61.90	1.80	62.00	1.80	62.00	1.80	61.90	1.80	61.90	1.80	61.90	1.80	62.00	0.76 ± 0.0006
train3	1.90	58.30	8.25	79.55	1.80	57.65	1.70	59.40	3.00	64.05	4.95	68.80	1.85	64.35	4.75	67.80	5.85	70.75	2.45	62.65	0.77 ± 0.0378

7.5.1. Training on Distinct MNIST Subsets

For this experiment, we selected Model 3 (`[196, 64, 196]`), which demonstrated the best performance in the previous test scenario. Each device received a distinct subset of the MNIST dataset, and, after 10 local training epochs, federated averaging was performed.

The results confirm that the federated model achieves convergence with improved performance compared to centralized training, highlighting the effectiveness of the federated aggregation process. In this scenario, models go through four federated rounds. Table 7 provides a detailed overview of the anomaly detection rates and F1 scores for digit '0', which is representative of the overall trend as the results for the other digits remain similar.

Table 7. Anomaly detection performance for digit 0, Model 3 `[196, 64, 196]`.

Training/ Federated Phase	Final Training Loss	Anomaly Detection Rate (%)						F1 Score
		MNIST	MNIST-C Fog	MNIST-C Motion Blur	MNIST-C Dotted Line	MNIST-C Spatter	EMNIST	
train0	0.0098	12.65	95.25	14.90	66.85	25.25	85.25	0.8615
fed0	-	9.50	93.95	11.05	51.55	18.45	84.80	**0.8729**
train1	0.0051	1.70	94.45	3.85	17.10	2.55	60.15	0.7433
fed1	-	0.90	94.40	2.25	11.70	1.20	56.15	0.7151
train2	0.0037	0.80	94.45	2.40	7.60	0.95	49.35	0.6573
fed2	-	0.50	94.50	1.40	5.55	0.50	45.70	0.6252
train3	0.0034	0.65	94.45	1.95	5.40	0.75	45.15	0.6193
fed3	-	0.50	94.40	0.95	4.90	0.50	42.55	0.5949
train4	0.0036	0.60	94.50	1.60	5.05	0.70	43.75	0.6062

The best-performing global model, obtained after the first federated round (**Federated 0**), achieves an F1 score of 0.8729, which surpasses the 0.8689 score obtained in the centralized setting. Interestingly, the subsequent federated rounds result in a gradual decline in the F1 score. However, a closer analysis reveals that this decline coincides with improved training loss and lower AD rates across all the test sets, including MNIST, EMNIST, and MNIST-C (excluding the "fog" variant). This indicates that the model's reconstruction capability is improving, even though the F1 score suggests otherwise.

Although counterintuitive at first glance, the decline in F1 score reflects the broader generalization power gained through FL. By sharing diverse local representations, the aggregated model becomes better at reconstructing previously unseen variations in digit structure, particularly beneficial for EMNIST and distorted MNIST-C examples. The exception is the "fog" variant, where the visual characteristics diverge significantly from MNIST (e.g., grayscale textures), limiting the model's ability to generalize due to the lack of such patterns in the training data.

While reconstruction quality improves, prioritizing AD performance (measured by the F1 score) introduces a trade-off: extended federated training may lead to overgeneralization and reduced discriminative power. Thus, in this scenario, a single round of federated learning is sufficient to obtain a high-performing global model, balancing reconstruction ability and AD accuracy. This observation that the final models selected by the current early stopping criterion may not always achieve the best AD performance highlights a key limitation in the training process. To address this, several practical solutions can be implemented.

Specifically, our current dual-phase early stopping method relies solely on the training loss as the stopping criterion, without directly considering AD performance. While this approach simplifies the stopping decision during local training, it could be enhanced by incorporating AD performance into the stopping criteria. However, tracking such metrics locally would increase computational overhead and power consumption.

Moreover, due to limited memory resources on embedded devices like the ESP32, saving multiple best-performing model checkpoints locally is not feasible. However, this challenge can be addressed by leveraging the central server in the FL setup. Devices can send relevant performance metrics, such as AD rates, to the server, which can then track and store the best global model based on these metrics. At the end of training, the server redistributes the best-performing model back to all devices. This approach enables better model selection aligned with AD goals without increasing memory demands on the devices themselves.

7.5.2. Robustness to Anomalous Training Data

To assess the robustness of the FL framework when exposed to anomalous training data, we conducted experiments where certain devices were trained entirely on corrupted versions of the dataset. Specifically, we focused on the *fog* corruption type from MNIST-C as prior experiments showed it produces the highest anomaly rates due to its substantial visual deviation from MNIST digits and the AE's difficulty in accurate reconstruction.

For this experiment, we again used Model 3 with the same architecture consisting of layers `[196, 64, 196]`, with all the same training configuration parameters.

In the following experiments, 1 to 5 out of 10 devices were trained on fog-corrupted digits, while the remaining devices used clean MNIST data. To monitor the behavior of the model, we focused on digit 9 as a representative case since it was consistently assigned to the corrupted training set in all configurations. The device configurations were as follows:

- Configuration 1: devices 0–8 trained on clean MNIST; device 9 trained on fog.
- Configuration 2: devices 0–7 on MNIST; devices 8–9 on fog.
- Configuration 3: devices 0–6 on MNIST; devices 7–9 on fog.
- Configuration 4: devices 0–5 on MNIST; devices 6–9 on fog.
- Configuration 5: devices 0–4 on MNIST; devices 5–9 on fog.

In each configuration, the local models underwent a round of training followed by federated aggregation. To better understand model behavior, we introduce an additional F1 score metric based on the fog dataset rather than EMNIST. This F1 score is designed to optimize false positives on MNIST and false negatives on fog. This allows us to evaluate how devices trained on fog datasets influence AD performance, considering both EMNIST and the fog variant. For simplicity, F1 scores based on EMNIST and fog are referred to as F1-EMNIST and F1-fog, respectively.

Figure 8 shows six charts presenting F1-EMNIST and F1-fog scores for the mentioned configurations. The charts (a–e) illustrate F1 score trends as the number of devices trained on fog increases. These charts track performance on device 9, which is assigned to fog training in all configurations. For example, Figure 8a corresponds to Config. 1, (b) to Config. 2, and so forth. Figure 8e,f both represent Config. 5 but differ in the digit used for training: (e) trains on digit 9 (fog corrupted), while (f) trains on digit 0 (clean MNIST), demonstrating how corrupted devices impact those trained on clean data.

As expected, after each local training phase, models trained on fog begin to slowly lose their ability to recognize anomalies since fog images are treated as "normal" during training. However, after federated aggregation, the global model rapidly recovers its anomaly detection capability, illustrating the strength of collaborative learning.

In subfigures (a–e), the F1-EMNIST score remains relatively stable, while F1-fog scores show greater variability. In Figure 8a, the initial F1-fog score after the first training round (**Training 0**) is very low due to the fog training set, as expected. However, following federated aggregation, the global model improves substantially, rendering the impact of the fog-trained device negligible. Subsequent training rounds show a slight decline in F1-fog scores as the model attempts to better fit the fog data on the corrupted device,

but aggregation consistently restores performance. This pattern indicates that a single corrupted device does not significantly degrade overall system performance.

As the number of fog-trained devices increases, a stronger bias towards fog data emerges, noticeable starting with three fog devices. Larger fluctuations appear between local training and federated rounds since the model more easily adapts to the fog dataset. However, the aggregated models (**Federated *i***) generally maintain good anomaly detection performance. This robustness is partly because the fog dataset is visually distinct and more difficult to learn, preventing it from dominating the federated aggregation even when half of the devices use fog data. Consequently, federated learning demonstrates resilience to anomalous datasets in this scenario.

Regarding F1-EMNIST behavior, the number of fog devices also affects performance. With few fog devices, the models behave similarly to prior experiments without fog, where EMNIST reconstruction improves, resulting in lower F1-EMNIST scores. As the number of devices trained on fog increases, their models have greater difficulty reconstructing EMNIST digits, leading to higher variability in F1-EMNIST scores compared to MNIST-biased models, which perform better at reconstructing EMNIST. Therefore, the global models after each federated round do not exhibit a strong decline in F1-EMNIST performance. The F1-EMNIST values for the best-performing global models remain in the range of approximately 0.80 to 0.87 across all the configurations. When compared to the clean data scenario in Table 7, where the best F1-EMNIST score achieved was 0.8729, the performance drop remains modest.

Figure 8f shows the behavior of device 0 when five other devices in the network are trained on fog. Compared to device 9 in the same configuration (Configuration 5), we observe that both devices reach similar F1 scores at the points of federated aggregation but exhibit opposite trends during local training. This behavior is expected as device 0 trains on clean MNIST data and improves the ability to reconstruct MNIST, while device 9 trains on fog data and thus improves fog reconstruction. These opposing learning trajectories result in balanced performance after aggregation, illustrating the stabilizing effect of federated averaging across heterogeneous local models.

The average behavior across all the devices for each FL configuration is summarized in Table 8. The table presents the mean F1 scores and standard deviations for each FL round during the training. As the number of devices trained on fog-corrupted data increases, corresponding to higher configuration numbers, we observe a decline in the average F1-fog scores. Additionally, the standard deviation tends to increase, indicating larger discrepancies between the performance of clean and anomalous devices. This highlights the growing divergence in local model performance as data heterogeneity grows.

Overall, the results show that, even as the number of devices exposed to fog increases, the global model maintains strong AD performance after each federated round. Although F1-fog scores decline slightly from Configuration 1 to Configuration 5, the global model continues to perform reliably. For example, Configuration 1's final federated round yields an AD rate of around 0.6% for MNIST and 92.95% for MNIST-C fog, resulting in an F1-fog score of 0.96. Configuration 5, despite higher error rates (8.5% for MNIST and 87.85% for fog), still achieves an acceptable F1-fog score of 0.89.

While FL robustness is commonly achieved through techniques such as gradient clipping and client selection, which reduce the impact of anomalous updates, these methods often introduce computational overhead and rely on assumptions about anomaly detection or gradient thresholds. Gradient clipping limits the magnitude of updates to prevent a single client's gradients from disproportionately affecting the global model, while client selection methods attempt to identify and prioritize reliable clients based on data quality or behavior patterns.

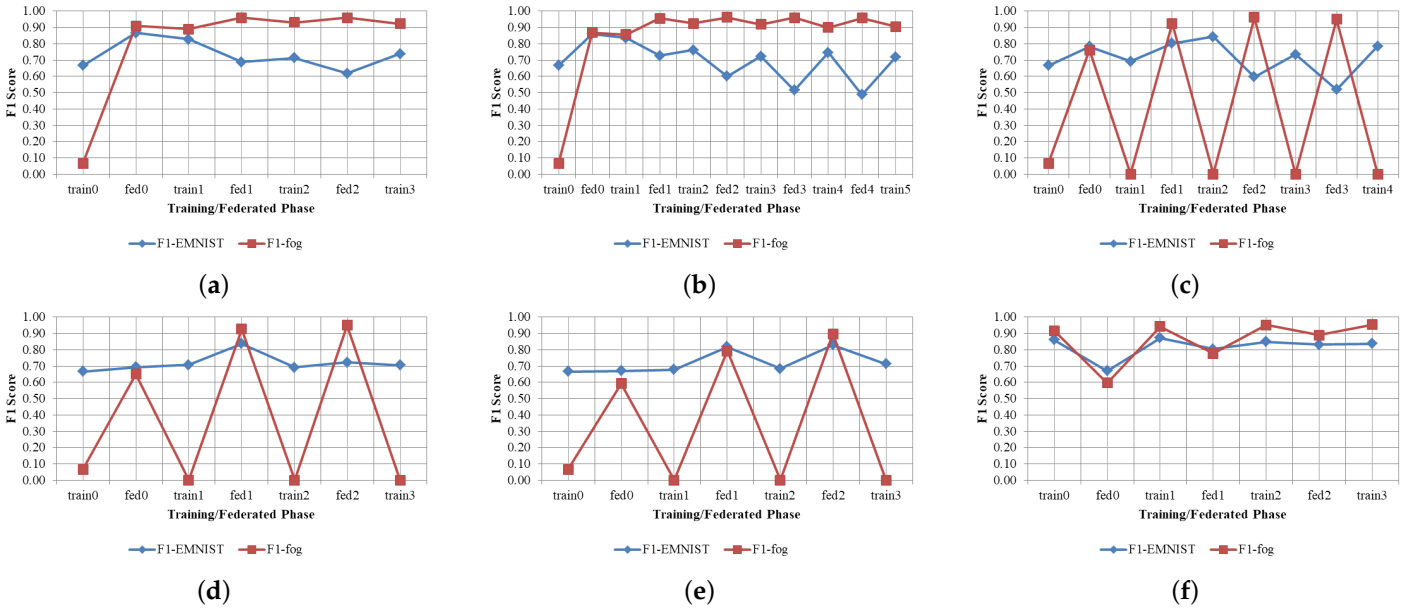


Figure 8. F1 score trends during federated training with varying numbers of devices trained on fog-corrupted data: (**a**) Configuration 1 (digit 9), (**b**) Config. 2 (digit 9), (**c**) Config. 3 (digit 9), (**d**) Config. 4 (digit 9), (**e**) Config. 5 (digit 9), and (**f**) Config. 5 (digit 0).

Table 8. Average F1-EMNIST and F1-fog scores across all devices for each FL configuration.

Training/ Federated Phase	Configuration 1		Configuration 2		Configuration 3		Configuration 4		Configuration 5	
	F1-EMNIST $\pm \sigma$	**F1-Fog** $\pm \sigma$	**F1-EMNIST** $\pm \sigma$	**F1-Fog** $\pm \sigma$	**F1-EMNIST** $\pm \sigma$	**F1-Fog** $\pm \sigma$	**F1-EMNIST** $\pm \sigma$	**F1-Fog** $\pm \sigma$	**F1-EMNIST** $\pm \sigma$	**F1-Fog** $\pm \sigma$
train0	0.8437 ± 0.0631	0.8382 ± 0.2710	0.8250 ± 0.0840	0.7530 ± 0.3593	0.8038 ± 0.0950	0.6651 ± 0.4158	0.7839 ± 0.1012	0.5783 ± 0.4456	0.7664 ± 0.1052	0.4927 ± 0.4558
fed0	0.8720 ± 0.0020	0.8996 ± 0.0038	0.8457 ± 0.0076	0.8467 ± 0.0113	0.7584 ± 0.0176	0.7385 ± 0.0167	0.6897 ± 0.0036	0.6492 ± 0.0018	0.6707 ± 0.0002	0.5943 ± 0.0027
train1	0.7726 ± 0.0218	0.9584 ± 0.0243	0.8012 ± 0.0212	0.9456 ± 0.0464	0.7790 ± 0.0626	0.6771 ± 0.4672	0.7808 ± 0.0821	0.5766 ± 0.4963	0.7675 ± 0.1012	0.4731 ± 0.4987
fed1	0.7381 ± 0.0176	0.9620 ± 0.0008	0.7627 ± 0.0190	0.9578 ± 0.0006	0.8157 ± 0.0092	0.9229 ± 0.0017	0.8417 ± 0.0043	0.9223 ± 0.0047	0.8111 ± 0.0075	0.7859 ± 0.0081
train2	0.7018 ± 0.0242	0.9646 ± 0.0119	0.7149 ± 0.0373	0.9638 ± 0.0203	0.7394 ± 0.0658	0.6846 ± 0.4724	0.7491 ± 0.0553	0.5840 ± 0.5026	0.7747 ± 0.0815	0.4788 ± 0.5047
fed2	0.6642 ± 0.0164	0.9659 ± 0.0019	0.6480 ± 0.0256	0.9653 ± 0.0023	0.6412 ± 0.0316	0.9666 ± 0.0029	0.7473 ± 0.0214	0.9511 ± 0.0008	0.8311 ± 0.0012	0.8936 ± 0.0041
train3	0.6498 ± 0.0326	0.9648 ± 0.0151	0.6558 ± 0.0453	0.9617 ± 0.0232	0.6473 ± 0.0592	0.6851 ± 0.4728	0.7368 ± 0.0354	0.5838 ± 0.5025	0.7993 ± 0.0610	0.4785 ± 0.5044
fed3	-	-	0.5729 ± 0.0303	0.9651 ± 0.0031	0.5584 ± 0.0268	0.9532 ± 0.0023	-	-	-	-
train4	-	-	0.6255 ± 0.0678	0.9558 ± 0.0314	0.6227 ± 0.1050	0.6832 ± 0.4714	-	-	-	-
fed4	-	-	0.5384 ± 0.0271	0.9635 ± 0.0030	-	-	-	-	-	-
train5	-	-	0.5976 ± 0.0661	0.9561 ± 0.0277	-	-	-	-	-	-

In contrast, our study evaluates the inherent resilience of the standard FedAvg aggregation method, which averages model updates from all devices. The robustness observed in our experiments derives from this aggregation mechanism itself. By combining updates, FedAvg dilutes the influence of any single corrupted or anomalous client. Thus, when most devices provide clean data, the aggregated global model is primarily shaped by reliable updates, reducing bias introduced by corrupted local models. This process allows gradual correction of biases introduced by anomalous updates, further enhancing resilience. Over successive FL rounds, this iterative averaging helps the global model to maintain robustness without added complexity.

7.6. Time Analysis of Federated Learning Workflow

In our experimental FL setup, the majority of the time is spent on local training performed by each client device. After completing local training for a number of epochs, clients send their model updates to the central server, which performs aggregation and returns the updated global model. Under stable Wi-Fi conditions, these communication steps are relatively fast, whereas local training is significantly more time-consuming.

For instance, in the non-IID setup (Table 6), training Model 3 on a dataset of 1000 images takes approximately 84 s per epoch, or about 840 s for 10 epochs. Since FL rounds are synchronous, the server must wait for all the clients to complete their local training before aggregation can proceed, meaning the total round duration depends on the slowest client. In this case, two FL rounds were needed to reach the best model, resulting in a total training time of around 2×850 s—roughly 30 min. Communication delays, such as sending updates to the server (uplink), server-side aggregation, and receiving the global model (downlink), were comparatively minimal. In this scenario, the uplink took around 30 milliseconds, aggregation about 20 milliseconds, and downlink another 30 milliseconds. These steps contribute negligibly to the overall round time. Across all the evaluated cases, local training remained the dominant factor in total FL runtime.

7.7. Comparison with Other Approaches

Table 9 summarizes the AD solutions designed for edge deployment, focusing on model types, device capabilities, and support for on-device training and FL. While many prior studies utilize AEs for efficient unsupervised AD, they mostly rely on more resource-rich and computationally superior hardware, such as Raspberry Pi, Jetson Nano, or desktop-class CPUs. Some works do target constrained hardware like ESP32 and STM32 MCUs [28,29], or Arduino Nano 33 BLE Sense [33], but these approaches typically focus on inference or incremental updates rather than full on-device training or FL. When on-device training or FL is performed, it usually involves more powerful platforms or simulated multi-device environments rather than real deployment on constrained devices.

In contrast, our work demonstrates full on-device training and inference on highly constrained real-world ESP32 MCUs. Compared to other studies, our work features one of the lowest memory requirements for model storage, ranging from 25 to 120 KB, which is substantially smaller than many related works that use models sized in megabytes (e.g., 12 MB in [26] or 29–42 MB in [42]) or omit memory details altogether. Despite the ESP32's limited resources, our models achieve competitive inference latencies between 9 and 42 ms, which is comparable to or better than the results reported on more capable edge devices, such as the Raspberry Pi 4 or Jetson Nano (13.1 and 19.6 ms, respectively [42]) Furthermore, unlike most approaches that use quantization or pruning to further reduce latency and memory footprint, we perform end-to-end training from scratch using standard 32-bit floating-point precision without simplifying the implementation and preserving performance at the same time.

Table 9. Comparison of anomaly detection systems on the edge.

Work	Device Type	Device (RAM/Frequency)	Model Type	Model Size	Inference Latency	On-Device Train	FL
[26]	-	-	SCVAE	12 MB	6 ms	-	-
[27]	-	-	Stacked AE	-	-	-	-
[28]	ESP32	520 KB/240 MHz	AE	-	-	-	-
[29]	STM32L476	128 KB/80 MHz	PCA, AE, Conv-AE	77.55 KB, -, -	6.428 ms, -, -	-	-
[30]	-	-	AE	-	-	-	-
[33]	Arduino Nano 33 BLE Sense	256 KB/64 MHz	AE	-	-	✓, OL	-
[34]	i5-7600K, ARM Cortex A72	-/3.8 GHz, 4 GB/1.5 GHz	Deep Conv-AE	2.7–273 KB	0.366–0.746 µs, 6.044–13.575 µs	✓	-
[40]	i7-11700K	64 GB/3.6 GHz	Deep AE for FL	-	-	✓	✓
[41]	i7-8700K, NVIDIA GeForce RTX 2080 Max-Q	32 GB/3.7 GHz, 8 GB GPU	LSTM AE	-	-	✓	✓
[42]	Raspberry Pi 4, Jetson Nano	4 GB/1.5 GHz, 4 GB GPU	AE, LSTM	29 MB, 42 MB	13.1 ms, 19.6 ms	✓	✓
[44]	-	-	WAFL-AE	-	-	✓	✓
[43]	-	-	AE, DNN	-	-	✓	✓
This work	ESP32-CAM	520 KB/240 MHz	AE	25–120 KB	9–42 ms	✓	✓

AE = autoencoder; Conv-AE = convolutional autoencoder; SCVAE = squeezed convolutional variational autoencoder; PCA = principal component analysis; WAFL = wireless ad hoc federated learning; DNN = deep neural network; LSTM = long short-term memory; OL = online learning; ✓ = feature present; "-" = not reported.

Furthermore, we extend this setup into a practical FL framework involving multiple physical ESP32 devices communicating over Wi-Fi. Unlike related FL studies that use more powerful hardware or simulate multi-device setups, our implementation supports real-world testbed decentralized learning under both IID and realistic non-IID data conditions. Through extensive evaluation across diverse data partitions and corruption levels, we demonstrate the feasibility and resilience of privacy-preserving decentralized learning on ultra-low-power embedded systems. Our solution stands out by enabling the full FL process on constrained devices, closing the gap between traditional edge-based AD, where training occurs centrally and only inference is performed on-device, and advanced collaborative learning, typically run on high-performance hardware.

8. Conclusions

This work demonstrates that both training and inference for AD can be performed entirely on resource-constrained edge hardware using 32-bit floating-point autoencoders. We designed and evaluated several lightweight models on the ESP32-CAM MCU, showing that reliable on-device training is feasible without relying on quantization, pruning, or pretraining. Despite its limited memory, the ESP32-CAM supports models with full training memory footprints between 100 and 480 KB, with many fitting entirely within the internal RAM, while datasets can be accommodated using the device's external 4 MB PSRAM. The system achieves training times between 12 and 210 min and inference latency ranging from 9 to 42 ms depending on model complexity. To balance training efficiency and model performance, we employed the proposed dual-phase early stopping strategy.

We further extended this setup into a practical FL testbed involving ten ESP32-CAM devices communicating over Wi-Fi. Through extensive experiments across diverse scenarios—including IID, non-IID, and corrupted data distributions—we demonstrated the system's robustness and scalability. The best-performing models achieved F1 scores of up to 0.87 with standard IID settings and 0.86 under the strict single-digit-per-device non-IID configuration. Importantly, even when up to two devices were completely trained on corrupted data, collaborative training remained stable without requiring explicit anomaly or malicious client detection. These results confirm that standard federated aggregation can effectively mitigate the influence of local data anomalies, featuring solid system resilience.

While our results demonstrate the feasibility and robustness of on-device training and FL on edge hardware, several limitations remain. The models and datasets used are intentionally small due to the strict memory constraints of the ESP32-CAM, and larger-scale applications have not yet been explored. As a result, the quality of AD is also constrained. Additionally, long training durations may limit applicability in battery-powered scenarios. Although our FL testbed involved ten devices, the upper scalability limit in terms of the number of participants, communication overhead, and convergence time remains untested. Furthermore, we assumed a stable network connection during the experiments, while real-world conditions often involve Wi-Fi signal degradation, packet loss, signal interruptions, or interference, which are not covered here. Evaluating the system under such conditions would provide a more realistic assessment of TinyFL in practical deployments.

Future work will focus on enabling continuous learning without needing to store datasets in memory, thereby freeing resources for larger models and allowing real-time adaptation. We aim to extend this approach using practical domain-specific data and address broader real-world challenges, such as unstable power supply during training (or battery-powered devices) and variable network conditions. Further research could explore system performance in environments with poor network connectivity (e.g., basements and long-distance links), providing insight into FL's capability under challenging conditions. In addition, we will conduct a deeper theoretical analysis of FL's resilience to corrupted data to further enhance system reliability. Finally, we plan to investigate deployments in non-vision domains such as time-series or sensor-based data, and evaluate optimization strategies to further reduce resource requirements.

Author Contributions: Conceptualization, A.T. and I.M.; methodology, A.T. and I.M.; software, A.T.; investigation, A.T.; resources, A.T.; data curation, A.T.; writing, A.T. and I.M.; supervision, I.M.; funding acquisition, I.M. All authors have read and agreed to the published version of the manuscript.

Funding: This research has been supported by the Serbian Ministry of Science, Technological Development and Innovation, through the Science and Technological Cooperation program Serbia–China, Research and development project No 00101957 2025 13440 003 000 620 021, and through the project "Scientific and Artistic Research Work of Researchers in Teaching and Associate Positions at the Faculty of Technical Sciences, University of Novi Sad 2025" (No. 01-50/295, Contract No. 451-03-137/2025-03/200156).

Data Availability Statement: Publicly available MNIST, EMNIST, and MNIST-C datasets were accessed via TensorFlow Datasets at https://www.tensorflow.org/datasets/catalog/mnist, https://www.tensorflow.org/datasets/catalog/emnist, and https://www.tensorflow.org/datasets/catalog/mnist_corrupted, respectively, accessed on 25 June 2025.

Conflicts of Interest: The authors declare no conflicts of interest.

Abbreviations

The following abbreviations are used in this manuscript:

ML	Machine Learning
FL	Federated Learning
IoT	Internet of Things
AE	Autoencoder
MCU	Microcontroller
AD	Anomaly Detection
PC	Personal Computer
IID	Independent and Identically Distributed
RAM	Random Access Memory
PSRAM	Pseudostatic RAM

References

1. David, R.; Duke, J.; Jain, A.; Janapa Reddi, V.; Jeffries, N.; Li, J.; Kreeger, N.; Nappier, I.; Natraj, M.; Wang, T.; et al. Tensorflow lite micro: Embedded machine learning for tinyml systems. *Proc. Mach. Learn. Syst.* **2021**, *3*, 800–811.
2. Abadade, Y.; Temouden, A.; Bamoumen, H.; Benamar, N.; Chtouki, Y.; Hafid, A.S. A Comprehensive Survey on TinyML. *IEEE Access* **2023**, *11*, 96892–96922. [CrossRef]
3. Tsoukas, V.; Gkogkidis, A.; Boumpa, E.; Kakarountas, A. A Review on the emerging technology of TinyML. *ACM Comput. Surv.* **2024**, *56*, 1–37. [CrossRef]
4. Rajapakse, V.; Karunanayake, I.; Ahmed, N. Intelligence at the Extreme Edge: A Survey on Reformable TinyML. *ACM Comput. Surv.* **2023**, *55*, 1–30. [CrossRef]
5. Ravindran, S. Cutting AI Down to Size. *Science* **2025**, *387*, 818–821. [CrossRef]
6. Prakash, S.; Stewart, M.; Banbury, C.; Mazumder, M.; Warden, P.; Plancher, B.; Reddi, V.J. Is TinyML Sustainable? *Commun. ACM* **2023**, *66*, 68–77. [CrossRef]
7. Vu, T.H.; Tu, N.H.; Huynh-The, T.; Lee, K.; Kim, S.; Voznak, M.; Pham, Q.V. Integration of TinyML and LargeML: A Survey of 6G and Beyond. *arXiv* **2025**, arXiv:2505.15854.
8. Trilles, S.; Hammad, S.S.; Iskandaryan, D. Anomaly detection based on artificial intelligence of things: A systematic literature mapping. *Internet Things* **2024**, *25*, 101063. [CrossRef]
9. Le, D.K.; Nguyen, X.L. Lightweight Unsupervised Model for Anomaly Detection on Microcontroller Platforms. *J. Marit. Res.* **2024**, *21*, 187–195.
10. Yap, Y.S.; Ahmad, M.R. Modified Overcomplete Autoencoder for Anomaly Detection Based on TinyML. *IEEE Sens. Lett.* **2024**, *8*, 1–4. [CrossRef]
11. Geyer, R.C.; Klein, T.; Nabi, M. Differentially private federated learning: A client level perspective. *arXiv* **2017**, arXiv:1712.07557.
12. Kairouz, P.; McMahan, H.B.; Avent, B.; Bellet, A.; Bennis, M.; Bhagoji, A.N.; Bonawitz, K.; Charles, Z.; Cormode, G.; Cummings, R.; et al. Advances and Open Problems in Federated Learning. *Found. Trends® Mach. Learn.* **2021**, *14*, 1–210. [CrossRef]
13. Zhang, T.; Gao, L.; He, C.; Zhang, M.; Krishnamachari, B.; Avestimehr, A.S. Federated learning for the internet of things: Applications, challenges, and opportunities. *IEEE Internet Things Mag.* **2022**, *5*, 24–29. [CrossRef]
14. da Silva, C.N.; Prazeres, C.V.S. Tiny Federated Learning for Constrained Sensors: A Systematic Literature Review. *IEEE Sens. Rev.* **2025**, *2*, 17–31. [CrossRef]
15. de Albuquerque Filho, J.E.; Brandao, L.C.; Fernandes, B.J.T.; Maciel, A.M. A review of neural networks for anomaly detection. *IEEE Access* **2022**, *10*, 112342–112367. [CrossRef]
16. Ghamry, F.M.; El-Banby, G.M.; El-Fishawy, A.S.; El-Samie, F.E.A.; Dessouky, M.I. A survey of anomaly detection techniques. *J. Opt.* **2024**, *53*, 756–774. [CrossRef]
17. Carannante, G.; Dera, D.; Aminul, O.; Bouaynaya, N.C.; Rasool, G. Self-assessment and robust anomaly detection with bayesian deep learning. In Proceedings of the 2022 25th International Conference on Information Fusion (FUSION), Linköping, Sweden, 4–7 July 2022; pp. 1–8.
18. Denouden, T.; Salay, R.; Czarnecki, K.; Abdelzad, V.; Phan, B.; Vernekar, S. Improving reconstruction autoencoder out-of-distribution detection with Mahalanobis distance. *arXiv* **2018**, arXiv:1812.02765.
19. Ruff, L.; Vandermeulen, R.A.; Franks, B.J.; Müller, K.R.; Kloft, M. Rethinking assumptions in deep anomaly detection. *arXiv* **2020**, arXiv:2006.00339.
20. Haider, Z.A.; Zeb, A.; Rahman, T.; Singh, S.K.; Akram, R.; Arishi, A.; Ullah, I. A Survey on anomaly detection in IoT: Techniques, challenges, and opportunities with the integration of 6G. *Comput. Netw.* **2025**, *270*, 111484. [CrossRef]

21. Capogrosso, L.; Cunico, F.; Cheng, D.S.; Fummi, F.; Cristani, M. A Machine Learning-Oriented Survey on Tiny Machine Learning. *IEEE Access* **2024**, *12*, 23406–23426. [CrossRef]
22. Zeeshan, M. Efficient Deep Learning Models for Edge IOT Devices-A Review. *Authorea Prepr.* **2024**. [CrossRef]
23. Naveen, S.; Kounte, M.R. Optimized Convolutional Neural Network at the IoT edge for image detection using pruning and quantization. *Multimed. Tools Appl.* **2025**, *84*, 5435–5455. [CrossRef]
24. Ghamari, S.; Ozcan, K.; Dinh, T.; Melnikov, A.; Carvajal, J.; Ernst, J.; Chai, S. Quantization-guided training for compact tinyml models. *arXiv* **2021**, arXiv:2103.06231.
25. Lin, J.; Chen, W.M.; Lin, Y.; Gohn, J.; Gan, C.; Han, S. Mcunet: Tiny deep learning on iot devices. *Adv. Neural Inf. Process. Syst.* **2020**, *33*, 11711–11722.
26. Kim, D.; Yang, H.; Chung, M.; Cho, S.; Kim, H.; Kim, M.; Kim, K.; Kim, E. Squeezed convolutional variational autoencoder for unsupervised anomaly detection in edge device industrial internet of things. In Proceedings of the 2018 International Conference on Information and Computer Technologies (ICICT), DeKalb, IL, USA, 23–25 March 2018; pp. 67–71.
27. Givnan, S.; Chalmers, C.; Fergus, P.; Ortega-Martorell, S.; Whalley, T. Anomaly detection using autoencoder reconstruction upon industrial motors. *Sensors* **2022**, *22*, 3166. [CrossRef]
28. Bratu, D.V.; Ilinoiu, R.Ș.T.; Cristea, A.; Zolya, M.A.; Moraru, S.A. Anomaly Detection Using Edge Computing AI on Low Powered Devices. In Proceedings of the IFIP International Conference on Artificial Intelligence Applications and Innovations, Crete, Greece, 17–20 June 2022; pp. 96–107.
29. Moallemi, A.; Burrello, A.; Brunelli, D.; Benini, L. Exploring scalable, distributed real-time anomaly detection for bridge health monitoring. *IEEE Internet Things J.* **2022**, *9*, 17660–17674. [CrossRef]
30. Luo, T.; Nagarajan, S.G. Distributed anomaly detection using autoencoder neural networks in WSN for IoT. In Proceedings of the 2018 IEEE International Conference on Communications (ICC), Kansas City, MO, USA, 20–24 May 2018; pp. 1–6.
31. Kwon, Y.D.; Li, R.; Venieris, S.I.; Chauhan, J.; Lane, N.D.; Mascolo, C. TinyTrain: Resource-aware task-adaptive sparse training of DNNs at the data-scarce edge. *arXiv* **2023**, arXiv:2307.09988.
32. Deutel, M.; Hannig, F.; Mutschler, C.; Teich, J. On-Device Training of Fully Quantized Deep Neural Networks on Cortex-M Microcontrollers. *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.* **2025**, *44*, 1250–1261. [CrossRef]
33. Ren, H.; Anicic, D.; Runkler, T.A. Tinyol: Tinyml with online-learning on microcontrollers. In Proceedings of the 2021 International Joint Conference on Neural Networks (IJCNN), Shenzhen, China, 18–22 July 2021; pp. 1–8.
34. Abbasi, S.; Famouri, M.; Shafiee, M.J.; Wong, A. OutlierNets: Highly compact deep autoencoder network architectures for on-device acoustic anomaly detection. *Sensors* **2021**, *21*, 4805. [CrossRef]
35. Qi, P.; Chiaro, D.; Piccialli, F. Small models, big impact: A review on the power of lightweight Federated Learning. *Future Gener. Comput. Syst.* **2024**, *162*, 107484. [CrossRef]
36. Kopparapu, K.; Lin, E.; Breslin, J.G.; Sudharsan, B. Tinyfedtl: Federated transfer learning on ubiquitous tiny iot devices. In Proceedings of the 2022 IEEE International Conference on Pervasive Computing and Communications Workshops and Other Affiliated Events (PerCom Workshops), Pisa, Italy, 21–25 March 2022; pp. 79–81.
37. Ficco, M.; Guerriero, A.; Milite, E.; Palmieri, F.; Pietrantuono, R.; Russo, S. Federated learning for IoT devices: Enhancing TinyML with on-board training. *Inf. Fusion* **2024**, *104*, 102189. [CrossRef]
38. Nikić, V.; Bortnik, D.; Lukić, M.; Vukobratović, D.; Mezei, I. Lightweight Digit Recognition in Smart Metering System Using Narrowband Internet of Things and Federated Learning. *Future Internet* **2024**, *16*, 402. [CrossRef]
39. Ren, H.; Anicic, D.; Runkler, T.A. TinyReptile: TinyML with federated meta-learning. In Proceedings of the 2023 International Joint Conference on Neural Networks (IJCNN), Gold Coast, Australia, 18–23 June 2023; pp. 1–9.
40. Novoa-Paradela, D.; Fontenla-Romero, O.; Guijarro-Berdiñas, B. Fast deep autoencoder for federated learning. *Pattern Recognit.* **2023**, *143*, 109805. [CrossRef]
41. Liu, X.; Su, X.; Campo, G.D.; Cao, J.; Fan, B.; Saavedra, E.; Santamaría, A.; Röning, J.; Hui, P.; Tarkoma, S. Federated Learning on 5G Edge for Industrial Internet of Things. *IEEE Netw.* **2025**, *39*, 289–297. [CrossRef]
42. Reis, M.J. Edge-FLGuard: A Federated Learning Framework for Real-Time Anomaly Detection in 5G-Enabled IoT Ecosystems. *Appl. Sci.* **2025**, *15*, 6452. [CrossRef]
43. Olanrewaju-George, B.; Pranggono, B. Federated learning-based intrusion detection system for the internet of things using unsupervised and supervised deep learning models. *Cyber Secur. Appl.* **2025**, *3*, 100068. [CrossRef]
44. Ochiai, H.; Nishihata, R.; Tomiyama, E.; Sun, Y.; Esaki, H. Detection of global anomalies on distributed iot edges with device-to-device communication. In Proceedings of the Twenty-Fourth International Symposium on Theory, Algorithmic Foundations, and Protocol Design for Mobile Networks and Mobile Computing, Washington, DC, USA, 23–26 October 2023; pp. 388–393.
45. Ai-Thinker Technology Co., Ltd. ESP32-CAM Schematic Diagram. Available online: https://docs.ai-thinker.com/_media/esp32/docs/esp32_cam_sch.pdf (accessed on 23 May 2025).
46. LeCun, Y.; Bottou, L.; Bengio, Y.; Haffner, P. Gradient-based learning applied to document recognition. *Proc. IEEE* **1998**, *86*, 2278–2324. [CrossRef]

47. Cohen, G.; Afshar, S.; Tapson, J.; van Schaik, A. EMNIST: Extending MNIST to handwritten letters. In Proceedings of the 2017 International Joint Conference on Neural Networks (IJCNN), Anchorage, AK, USA, 14–19 May 2017; pp. 2921–2926. [CrossRef]
48. Mu, N.; Gilmer, J. MNIST-C: A Robustness Benchmark for Computer Vision. *arXiv* **2019**, arXiv:1906.02337.
49. TensorFlow. TensorFlow Datasets. Available online: https://www.tensorflow.org/datasets (accessed on 23 May 2025).
50. TensorFlow. EarlyStopping Callback. Available online: https://www.tensorflow.org/api_docs/python/tf/keras/callbacks/EarlyStopping (accessed on 3 June 2025).
51. Abadi, M.; Barham, P.; Chen, J.; Chen, Z.; Davis, A.; Dean, J.; Devin, M.; Ghemawat, S.; Irving, G.; Isard, M.; et al. {TensorFlow}: A system for {Large-Scale} machine learning. In Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16), Savannah, GA, USA, 2–4 November 2016; pp. 265–283.
52. McMahan, B.; Moore, E.; Ramage, D.; Hampson, S.; y Arcas, B.A. Communication-efficient learning of deep networks from decentralized data. In Proceedings of the 20th International Conference on Artificial Intelligence and Statistics, Fort Lauderdale, FL, USA, 20–22 April 2017; pp. 1273–1282.
53. Benmalek, M.; Benrekia, M.A.; Challal, Y. Security of federated learning: Attacks, defensive mechanisms, and challenges. *Rev. Des Sci. Technol. L'Inf.-Série RIA Rev. D'Intelligence Artif.* **2022**, *36*, 49–59. [CrossRef]
54. Alsulaimawi, Z. Federated Learning with Anomaly Detection via Gradient and Reconstruction Analysis. *arXiv* **2024**, arXiv:2403.10000.
55. Allouah, Y.; Guerraoui, R.; Gupta, N.; Jellouli, A.; Rizk, G.; Stephan, J. Adaptive gradient clipping for robust federated learning. *arXiv* **2024**, arXiv:2405.14432.
56. Zhang, Z.; Cao, X.; Jia, J.; Gong, N.Z. Fldetector: Defending federated learning against model poisoning attacks via detecting malicious clients. In Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, Washington, DC, USA, 14–18 August 2022; pp. 2545–2555.
57. Tanović, A.; Mezei, I. Embedded Parallel K-Means Algorithm Evaluation on ESP32 Across Various Memory Allocations. In Proceedings of the 2024 IEEE East-West Design & Test Symposium (EWDTS), Yerevan, Armenia, 13–17 November 2024; pp. 1–7.